

Escape From Heidelberg Dynamics Godot

Generated by Doxygen 1.11.0

1 Game Systems, tested using Xunit	1
1.1 Overview	1
1.2 Setting Up the Environment	1
1.2.1 Using Nix	1
1.2.2 On Ubuntu	1
1.2.3 On Windows	1
1.2.4 Running Tests	2
1.3 Coding Style	2
1.4 Health.cs	2
2 Game Systems, tested using Xunit	3
2.1 Overview	3
2.2 Setting Up the Environment	3
2.2.1 Using Nix	3
2.2.2 On Ubuntu	3
2.2.3 On Windows	3
2.2.4 Running Tests	4
2.3 Coding Style	4
2.4 Health.cs	4
3 Namespace Index	5
3.1 Namespace List	5
4 Hierarchical Index	7
4.1 Class Hierarchy	7
5 Class Index	9
5.1 Class List	9
6 File Index	11
6.1 File List	11
7 Namespace Documentation	13
7.1 TestProject1 Namespace Reference	13
8 Class Documentation	15
8.1 BossHealth Class Reference	15
8.1.1 Detailed Description	16
8.1.2 Member Data Documentation	17
8.1.2.1 health	17
8.2 Bossturret Class Reference	17
8.2.1 Detailed Description	18
8.2.2 Member Function Documentation	18
8.2.2.1 _PhysicsProcess()	18
8.2.2.2 CheckLineOfSight()	19

8.2.2.3 Fire1()	19
8.2.2.4 Fire2()	19
8.3 DialogueManager Class Reference	20
8.3.1 Detailed Description	21
8.3.2 Member Function Documentation	21
8.3.2.1 _Ready()	21
8.3.2.2 DisplayDialogue()	21
8.3.2.3 EndDialogue()	21
8.3.2.4 StartDialogue()	22
8.4 explosion Class Reference	22
8.4.1 Detailed Description	23
8.4.2 Member Function Documentation	23
8.4.2.1 _Ready()	23
8.5 Health Class Reference	24
8.5.1 Detailed Description	25
8.5.2 Constructor & Destructor Documentation	26
8.5.2.1 Health() [1/2]	26
8.5.2.2 Health() [2/2]	26
8.5.3 Member Function Documentation	26
8.5.3.1 ApplyDamage() [1/2]	26
8.5.3.2 ApplyDamage() [2/2]	27
8.5.3.3 ApplyTrueDamage() [1/2]	27
8.5.3.4 ApplyTrueDamage() [2/2]	27
8.5.3.5 damageHP() [1/2]	27
8.5.3.6 damageHP() [2/2]	28
8.5.3.7 damageShield() [1/2]	28
8.5.3.8 damageShield() [2/2]	28
8.5.3.9 DestroyShield() [1/2]	28
8.5.3.10 DestroyShield() [2/2]	28
8.5.3.11 HealBoth() [1/2]	29
8.5.3.12 HealBoth() [2/2]	29
8.5.3.13 HealHP() [1/2]	29
8.5.3.14 HealHP() [2/2]	29
8.5.3.15 HealShield() [1/2]	29
8.5.3.16 HealShield() [2/2]	30
8.5.3.17 RestoreHealth() [1/2]	30
8.5.3.18 RestoreHealth() [2/2]	30
8.5.3.19 RestoreShield() [1/2]	30
8.5.3.20 RestoreShield() [2/2]	30
8.5.3.21 Revive() [1/2]	30
8.5.3.22 Revive() [2/2]	31
8.5.4 Member Data Documentation	31

8.5.4.1 _damageBias	31
8.5.5 Property Documentation	31
8.5.5.1 _alive	31
8.5.5.2 _hp	31
8.5.5.3 _hpMax	31
8.5.5.4 _resistanceElectric	31
8.5.5.5 _resistancePhysical	32
8.5.5.6 _shield	32
8.5.5.7 _shieldMax	32
8.5.5.8 _vulnElectric	32
8.5.5.9 _vulnPhysical	32
8.6 InteractionArea Class Reference	33
8.6.1 Detailed Description	34
8.6.2 Member Function Documentation	35
8.6.2.1 _Ready()	35
8.6.2.2 DisplayDialogue()	35
8.6.2.3 EndDialogue()	35
8.6.2.4 GetInteractType()	35
8.6.2.5 GetLabel()	35
8.6.2.6 GetValue()	36
8.6.2.7 SetInteractType()	36
8.6.2.8 SetLabel()	36
8.6.2.9 SetValue()	36
8.6.2.10 StartDialogue()	36
8.7 level_4 Class Reference	37
8.7.1 Detailed Description	38
8.7.2 Member Function Documentation	38
8.7.2.1 _Ready()	38
8.8 Player Class Reference	39
8.8.1 Detailed Description	40
8.8.2 Member Function Documentation	41
8.8.2.1 _PhysicsProcess()	41
8.8.2.2 _Ready()	41
8.8.2.3 EndGame()	42
8.8.2.4 ExecuteInteract()	42
8.8.2.5 Fire()	42
8.8.2.6 GetPosition()	43
8.8.2.7 IncreaseBossCount()	43
8.8.2.8 SetExtraCard()	43
8.8.2.9 SetMoveable()	43
8.8.2.10 UpdateAnimation()	44
8.9 PlayerHealth Class Reference	44

8.9.1 Detailed Description	45
8.9.2 Member Data Documentation	46
8.9.2.1 health	46
8.10 projectile Class Reference	46
8.10.1 Detailed Description	47
8.10.2 Member Function Documentation	47
8.10.2.1 _PhysicsProcess()	47
8.10.2.2 _Ready()	47
8.11 Turret Class Reference	48
8.11.1 Detailed Description	49
8.11.2 Member Function Documentation	49
8.11.2.1 _PhysicsProcess()	49
8.11.2.2 _Ready()	49
8.11.2.3 CheckLineOfSight()	49
8.11.2.4 Fire()	50
8.12 TurretHealth Class Reference	50
8.12.1 Detailed Description	51
8.12.2 Member Data Documentation	52
8.12.2.1 health	52
8.13 TestProject1.UnitTest1 Class Reference	52
8.13.1 Detailed Description	53
8.13.2 Constructor & Destructor Documentation	53
8.13.2.1 UnitTest1()	53
8.13.3 Member Function Documentation	53
8.13.3.1 ApplyDamage_Test()	53
8.13.3.2 HealBoth_Test()	54
8.13.3.3 Health_Initialization_Test()	54
8.13.3.4 True_Damage_Test()	54
9 File Documentation	57
9.1 BossHealth.cs	57
9.2 Bossturret.cs	57
9.3 NinePatchRect.cs	58
9.4 Player.cs	58
9.5 PlayerHealth.cs	61
9.6 Turret.cs	61
9.7 TurretHealth.cs	62
9.8 GlobalUsings.cs	62
9.9 Health.cs	62
9.10 Health.cs	63
9.11 UnitTest1.cs	64
9.12 InteractionArea.cs	66

9.13 level_4.cs	67
9.14 explosion.cs	67
9.15 projectile.cs	67
Index	69

Chapter 1

Game Systems, tested using Xunit

1.1 Overview

This project includes a basic health system implemented in C# with tests written using XUnit.

1.2 Setting Up the Environment

1.2.1 Using Nix

1. **Install Nix:** Follow the instructions at <https://nixos.org/download.html>.
2. **Enter Nix Shell:** Run the following command in the project directory to enter the Nix environment with Godot 4 and .NET SDK 8.
`nix-shell`

1.2.2 On Ubuntu

If you prefer to set up .NET SDK 8 directly on Ubuntu without using Nix:

1. **Install the Microsoft package repository and GPG keys:**

```
wget https://packages.microsoft.com/config/ubuntu/20.04/packages-microsoft-prod.deb -O packages-microsoft-prod.deb
sudo dpkg -i packages-microsoft-prod.deb
sudo apt-get update
sudo apt-get install -y dotnet-sdk-8.0
```

1.2.3 On Windows

Install Nix on WSL.

1.2.4 Running Tests

To run the XUnit tests in the `TestProject1` directory, use the following command:

```
dotnet test TestProject1.sln
```

This will build the project and execute all tests, providing a summary of the test results.

This setup and guide are intended for internal use to streamline the development and testing process for the game systems project.

1.3 Coding Style

The Coding Style is based on the MS .NET C# coding style:

<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

Class Names and Namespaces are named in PascalCase.

Functions, Parameters, and Variables of any kind are to be named in camelCase.

Both public and private variables are to be prefixed with an underscore.

Example:

```
using System;

public class Health
{
    private int _hp;

    // Constructor to initialize health with a starting value
    public Health(int initialHp)
    {
        _hp = initialHp;
    }

    // Function to apply damage, using Math.Max to ensure hp doesn't go below 0
    public void Damage(int amount)
    {
        _hp = Math.Max(0, _hp - amount);
    }

    // Function to apply healing, using Math.Min to ensure hp doesn't exceed a maximum value (e.g., 100)
    public void Heal(int amount)
    {
        int maxHealth = 100; // You can modify this value based on your requirements
        _hp = Math.Min(maxHealth, _hp + amount);
    }

    // Function to get the current health value
    public int GetHealth()
    {
        return _hp;
    }
}
```

1.4 Health.cs

Health.cs contains the health system.

Entities have a separate `Health` and `Shield`, the shield gets depleted and regenerated more easily than health, and once health drops to zero the entity dies, even if it has shield points left.

All calculations are to be made in integer form for speed, shifted one digit to the left. e.g. 852 equals 85.2f

Chapter 2

Game Systems, tested using Xunit

2.1 Overview

This project includes a basic health system implemented in C# with tests written using XUnit.

2.2 Setting Up the Environment

2.2.1 Using Nix

1. **Install Nix:** Follow the instructions at <https://nixos.org/download.html>.
2. **Enter Nix Shell:** Run the following command in the project directory to enter the Nix environment with Godot 4 and .NET SDK 8.
`nix-shell`

2.2.2 On Ubuntu

If you prefer to set up .NET SDK 8 directly on Ubuntu without using Nix:

1. **Install the Microsoft package repository and GPG keys:**
`wget https://packages.microsoft.com/config/ubuntu/20.04/packages-microsoft-prod.deb -O packages-microsoft-prod.deb`
`sudo dpkg -i packages-microsoft-prod.deb`
`sudo apt-get update`
`sudo apt-get install -y dotnet-sdk-8.0`

2.2.3 On Windows

Install Nix on WSL.

2.2.4 Running Tests

To run the XUnit tests in the `TestProject1` directory, use the following command:

```
dotnet test TestProject1.sln
```

This will build the project and execute all tests, providing a summary of the test results.

This setup and guide are intended for internal use to streamline the development and testing process for the game systems project.

2.3 Coding Style

The Coding Style is based on the MS .NET C# coding style:

<https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

Class Names and Namespaces are named in PascalCase.

Functions, Parameters, and Variables of any kind are to be named in camelCase.

Both public and private variables are to be prefixed with an underscore.

Example:

```
using System;

public class Health
{
    private int _hp;

    // Constructor to initialize health with a starting value
    public Health(int initialHp)
    {
        _hp = initialHp;
    }

    // Function to apply damage, using Math.Max to ensure hp doesn't go below 0
    public void Damage(int amount)
    {
        _hp = Math.Max(0, _hp - amount);
    }

    // Function to apply healing, using Math.Min to ensure hp doesn't exceed a maximum value (e.g., 100)
    public void Heal(int amount)
    {
        int maxHealth = 100; // You can modify this value based on your requirements
        _hp = Math.Min(maxHealth, _hp + amount);
    }

    // Function to get the current health value
    public int GetHealth()
    {
        return _hp;
    }
}
```

2.4 Health.cs

Health.cs contains the health system.

Entities have a separate `Health` and `Shield`, the shield gets depleted and regenerated more easily than health, and once health drops to zero the entity dies, even if it has shield points left.

All calculations are to be made in integer form for speed, shifted one digit to the left. e.g. 852 equals 85.2f

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

TestProject1	13
--	----

Chapter 4

Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

AnimatedSprite2D	
explosion	22
Area2D	
InteractionArea	33
CharacterBody2D	
Bossturret	17
Player	39
Turret	48
Health	24
Node2D	
BossHealth	15
DialogueManager	20
PlayerHealth	44
TurretHealth	50
level_4	37
RigidBody2D	
projectile	46
TestProject1.UnitTest1	52

Chapter 5

Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

BossHealth	15
Bossturret	17
DialogueManager	20
explosion	22
Health	24
InteractionArea	33
level_4	37
Player	39
PlayerHealth	44
projectile	46
Turret	48
TurretHealth	50
TestProject1.UnitTest1	52

Chapter 6

File Index

6.1 File List

Here is a list of all documented files with brief descriptions:

InteractionArea.cs	66
characters/ BossHealth.cs	57
characters/ Bossturret.cs	57
characters/ NinePatchRect.cs	58
characters/ Player.cs	58
characters/ PlayerHealth.cs	61
characters/ Turret.cs	61
characters/ TurretHealth.cs	62
game-systems/TestProject1/ GlobalUsings.cs	62
game-systems/TestProject1/ Health.cs	62
game-systems/TestProject1/ UnitTest1.cs	64
levels/ level_4.cs	67
projectiles/ explosion.cs	67
projectiles/ projectile.cs	67
systems/TestProject1/ Health.cs	63

Chapter 7

Namespace Documentation

7.1 TestProject1 Namespace Reference

Classes

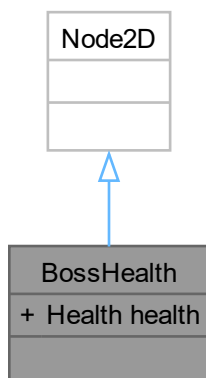
- class [UnitTest1](#)

Chapter 8

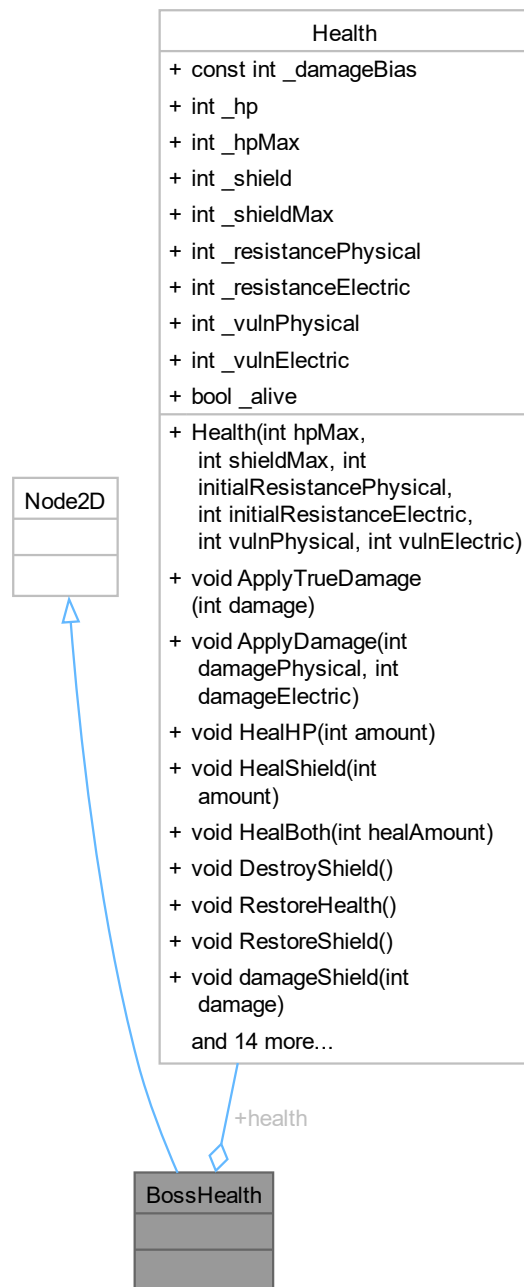
Class Documentation

8.1 BossHealth Class Reference

Inheritance diagram for BossHealth:



Collaboration diagram for BossHealth:



Public Attributes

- `Health health = new Health(1000,1000,100,100,100,100)`

8.1.1 Detailed Description

Definition at line 4 of file [BossHealth.cs](#).

8.1.2 Member Data Documentation

8.1.2.1 health

```
Health BossHealth.health = new Health(1000,1000,100,100,100,100)
```

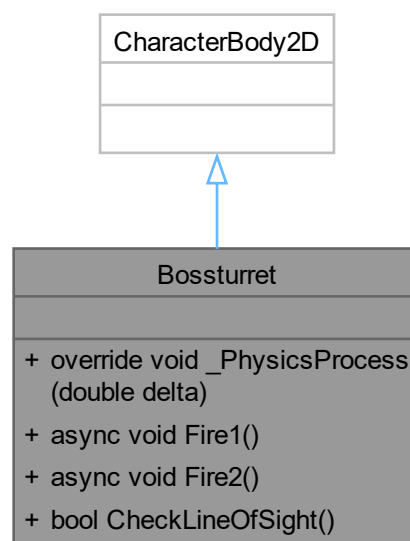
Definition at line 6 of file [BossHealth.cs](#).

The documentation for this class was generated from the following file:

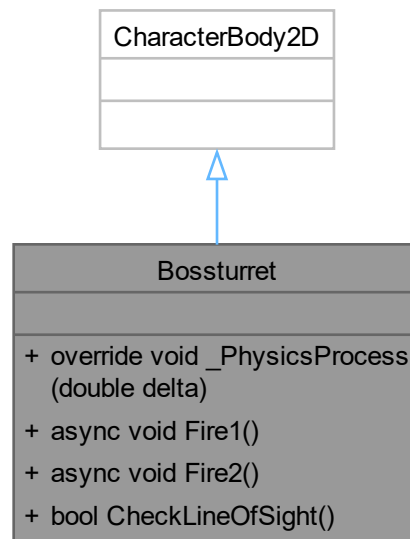
- [characters/BossHealth.cs](#)

8.2 Bossturret Class Reference

Inheritance diagram for Bossturret:



Collaboration diagram for Bossturret:



Public Member Functions

- override void `_PhysicsProcess` (double delta)
- async void `Fire1` ()
- async void `Fire2` ()
- bool `CheckLineOfSight` ()

8.2.1 Detailed Description

Definition at line 4 of file [Bossturret.cs](#).

8.2.2 Member Function Documentation

8.2.2.1 `_PhysicsProcess`()

```

override void Bossturret._PhysicsProcess (
    double delta) [inline]
  
```

Definition at line 13 of file [Bossturret.cs](#).

```

00013                                     {
00014     if(!GetNode<BossHealth>("Health").health._alive){
00015         hasWeapon = false;
00016         if(!counter){
00017             GetParent().GetNode<Player>("Player").IncreaseBossCount();
00018             counter = true;
00019         }
00020         GetNode<Sprite2D>("Turret").Visible = false;
00021     }
  
```

```

00022         seesPlayer = CheckLineOfSight();
00023         if (hasWeapon) {
00024             LookAt(GetParent().GetNode<Player>("Player").GetPosition());
00025             if (seesPlayer && canFire) {
00026                 Fire1();
00027                 Fire2();
00028             }
00029         }
00030     }

```

8.2.2.2 CheckLineOfSight()

```
bool Bossturret.CheckLineOfSight () [inline]
```

Definition at line 53 of file [Bossturret.cs](#).

```

00053         {
00054             Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00055             double distance = Mathf.Sqrt((playerPos.X - Position.X)*(playerPos.X - Position.X) +
00056                 (playerPos.Y - Position.Y)*(playerPos.Y - Position.Y));
00057             if (distance < 300) {
00058                 return true;
00059             }
00060             return false;
00061         }

```

8.2.2.3 Fire1()

```
async void Bossturret.Fire1 () [inline]
```

Definition at line 32 of file [Bossturret.cs](#).

```

00032         {
00033             GetNode<AudioStreamPlayer>("BulletSound").Play();
00034             RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00035             GetParent().AddChild(bulletInstance);
00036             bulletInstance.Position = GetNode<Node2D>("BulletPoint1").GlobalPosition;
00037             bulletInstance.RotationDegrees = RotationDegrees;
00038             bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00039             canFire = false;
00040             await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00041             canFire = true;
00042         }

```

8.2.2.4 Fire2()

```
async void Bossturret.Fire2 () [inline]
```

Definition at line 43 of file [Bossturret.cs](#).

```

00043         {
00044             await ToSignal(GetTree().CreateTimer(0.35), "timeout");
00045             GetNode<AudioStreamPlayer>("BulletSound").Play();
00046             RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00047             GetParent().AddChild(bulletInstance);
00048             bulletInstance.Position = GetNode<Node2D>("BulletPoint2").GlobalPosition;
00049             bulletInstance.RotationDegrees = RotationDegrees;
00050             bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00051         }

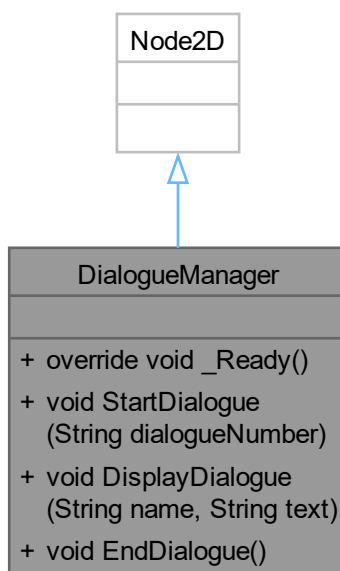
```

The documentation for this class was generated from the following file:

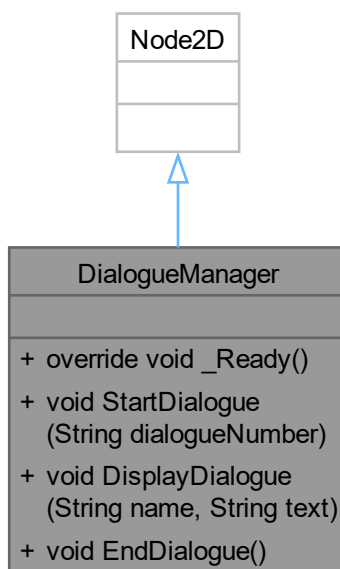
- `characters/Bossturret.cs`

8.3 DialogueManager Class Reference

Inheritance diagram for DialogueManager:



Collaboration diagram for DialogueManager:



Public Member Functions

- override void [_Ready](#) ()
- void [StartDialogue](#) (String dialogueNumber)
- void [DisplayDialogue](#) (String name, String text)
- void [EndDialogue](#) ()

8.3.1 Detailed Description

Definition at line 4 of file [NinePatchRect.cs](#).

8.3.2 Member Function Documentation

8.3.2.1 [_Ready\(\)](#)

```
override void DialogueManager._Ready () [inline]
```

Definition at line 7 of file [NinePatchRect.cs](#).

```
00007         {
00008             Visible = true;
00009         }
```

8.3.2.2 [DisplayDialogue\(\)](#)

```
void DialogueManager.DisplayDialogue (
    String name,
    String text) [inline]
```

Definition at line 30 of file [NinePatchRect.cs](#).

```
00030         {
00031             GetNode<RichTextLabel>("Name").Text = name;
00032             GetNode<RichTextLabel>("Text").Text = text;
00033         }
```

8.3.2.3 [EndDialogue\(\)](#)

```
void DialogueManager.EndDialogue () [inline]
```

Definition at line 35 of file [NinePatchRect.cs](#).

```
00035         {
00036             Visible = false;
00037             //GetParent().SetMoveable(true);
00038         }
```

8.3.2.4 StartDialogue()

```
void DialogueManager.StartDialogue (
    String dialogueNumber) [inline]
```

Definition at line 12 of file [NinePatchRect.cs](#).

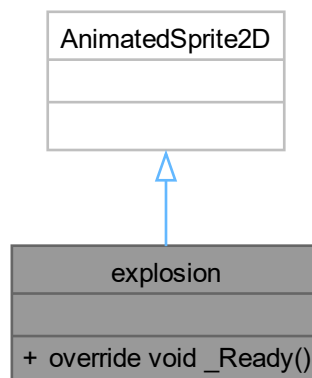
```
00012                                     {
00013     //GetParent().SetMoveable(false);
00014     Visible = false;
00015     switch(dialogueNumber){
00016     case "0":
00017         DisplayDialogue("Game","Welcome to Heidelberg Dynamics! You are L4lk4, and need to
00018     escape this facility." + " Use [W][A][S][D] to move to the next room. Press [E] to
00019     continue");
00019         break;
00020     case "1":
00021         DisplayDialogue("Game","You need to be careful. This facility is full of enemy
00022         turrets that will" + " shoot you on sight. Use [LMB] to shoot your weapon");
00023         break;
00024     default:
00025         EndDialogue();
00026         break;
00027     }
00028 }
```

The documentation for this class was generated from the following file:

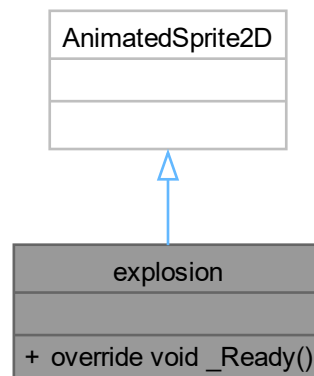
- characters/NinePatchRect.cs

8.4 explosion Class Reference

Inheritance diagram for explosion:



Collaboration diagram for explosion:



Public Member Functions

- `override void \_Ready ()`

8.4.1 Detailed Description

Definition at line 4 of file [explosion.cs](#).

8.4.2 Member Function Documentation

8.4.2.1 `_Ready()`

```
override void explosion._Ready () [inline]
```

Definition at line 6 of file [explosion.cs](#).

```
00006      {
00007          Position = new Vector2(0,0);
00008          //Position = GetParent().GlobalPosition;
00009      }
```

The documentation for this class was generated from the following file:

- `projectiles/explosion.cs`

8.5 Health Class Reference

Collaboration diagram for Health:

Health
+ const int _damageBias + int _hp + int _hpMax + int _shield + int _shieldMax + int _resistancePhysical + int _resistanceElectric + int _vulnPhysical + int _vulnElectric + bool _alive
+ Health(int hpMax, int shieldMax, int initialResistancePhysical, int initialResistanceElectric, int vulnPhysical, int vulnElectric) + void ApplyTrueDamage(int damage) + void ApplyDamage(int damagePhysical, int damageElectric) + void HealHP(int amount) + void HealShield(int amount) + void HealBoth(int healAmount) + void DestroyShield() + void RestoreHealth() + void RestoreShield() + void damageShield(int damage) - and 14 more...

Public Member Functions

- [Health](#) (int hpMax, int shieldMax, int initialResistancePhysical, int initialResistanceElectric, int vulnPhysical, int vulnElectric)
- void [ApplyTrueDamage](#) (int damage)

- void [ApplyDamage](#) (int damagePhysical, int damageElectric)
- void [HealHP](#) (int amount)
- void [HealShield](#) (int amount)
- void [HealBoth](#) (int healAmount)
- void [DestroyShield](#) ()
- void [RestoreHealth](#) ()
- void [RestoreShield](#) ()
- void [damageShield](#) (int damage)
- void [damageHP](#) (int damage)
- void [Revive](#) ()
- [Health](#) (int hpMax, int shieldMax, int initialResistancePhysical, int initialResistanceElectric, int vulnPhysical, int vulnElectric)
- void [ApplyTrueDamage](#) (int damage)
- void [ApplyDamage](#) (int damagePhysical, int damageElectric)
- void [HealHP](#) (int amount)
- void [HealShield](#) (int amount)
- void [HealBoth](#) (int healAmount)
- void [DestroyShield](#) ()
- void [RestoreHealth](#) ()
- void [RestoreShield](#) ()
- void [damageShield](#) (int damage)
- void [damageHP](#) (int damage)
- void [Revive](#) ()

Static Public Attributes

- const int [_damageBias](#) = 12

Properties

- int [_hp](#) [get]
- int [_hpMax](#) [get]
- int [_shield](#) [get]
- int [_shieldMax](#) [get]
- int [_resistancePhysical](#) [get]
- int [_resistanceElectric](#) [get]
- int [_vulnPhysical](#) [get]
- int [_vulnElectric](#) [get]
- bool [_alive](#) = true [get]

8.5.1 Detailed Description

Definition at line 3 of file [Health.cs](#).

8.5.2 Constructor & Destructor Documentation

8.5.2.1 Health() [1/2]

```
Health.Health (
    int hpMax,
    int shieldMax,
    int initialResistancePhysical,
    int initialResistanceElectric,
    int vulnPhysical,
    int vulnElectric) [inline]
```

Definition at line 16 of file [Health.cs](#).

```
00017 {
00018     _hpMax = hpMax;
00019     _shieldMax = shieldMax;
00020     _hp = _hpMax;
00021     _shield = _hpMax;
00022     _resistancePhysical = initialResistancePhysical;
00023     _resistanceElectric = initialResistanceElectric;
00024     _vulnElectric = vulnElectric;
00025     _vulnPhysical = vulnPhysical;
00026 }
```

8.5.2.2 Health() [2/2]

```
Health.Health (
    int hpMax,
    int shieldMax,
    int initialResistancePhysical,
    int initialResistanceElectric,
    int vulnPhysical,
    int vulnElectric) [inline]
```

Definition at line 16 of file [Health.cs](#).

```
00017 {
00018     _hpMax = hpMax;
00019     _shieldMax = shieldMax;
00020     _hp = _hpMax;
00021     _shield = _hpMax;
00022     _resistancePhysical = initialResistancePhysical;
00023     _resistanceElectric = initialResistanceElectric;
00024     _vulnElectric = vulnElectric;
00025     _vulnPhysical = vulnPhysical;
00026 }
```

8.5.3 Member Function Documentation

8.5.3.1 ApplyDamage() [1/2]

```
void Health.ApplyDamage (
    int damagePhysical,
    int damageElectric) [inline]
```

Definition at line 40 of file [Health.cs](#).

```
00041 {
00042     // line-by-line for easy debugging
00043     // doing stuff just twice doesn't warrant its own function
00044
00045     int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00046     int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00047
00048     int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00049     int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00050
00051     int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00052
00053     ApplyTrueDamage(damageTotal);
00054 }
```

8.5.3.2 ApplyDamage() [2/2]

```
void Health.ApplyDamage (
    int damagePhysical,
    int damageElectric) [inline]
```

Definition at line 40 of file [Health.cs](#).

```
00041     {
00042         // line-by-line for easy debugging
00043         // doing stuff just twice down's't warrant its own function
00044
00045         int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00046         int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00047
00048         int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00049         int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00050
00051         int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00052
00053         ApplyTrueDamage(damageTotal);
00054     }
```

8.5.3.3 ApplyTrueDamage() [1/2]

```
void Health.ApplyTrueDamage (
    int damage) [inline]
```

Definition at line 28 of file [Health.cs](#).

```
00029     {
00030         int damageAfterShield = Math.Max(damage - _shield, 0);
00031         _shield = Math.Max(_shield - damage, 0);
00032         _hp = _hp - damageAfterShield;
00033
00034         if (_hp <= 0)
00035         {
00036             _hp = 0;
00037             _alive = false;
00038         }
00039     }
```

8.5.3.4 ApplyTrueDamage() [2/2]

```
void Health.ApplyTrueDamage (
    int damage) [inline]
```

Definition at line 28 of file [Health.cs](#).

```
00029     {
00030         int damageAfterShield = Math.Max(damage - _shield, 0);
00031         _shield = Math.Max(_shield - damage, 0);
00032         _hp = _hp - damageAfterShield;
00033
00034         if (_hp <= 0)
00035         {
00036             _hp = 0;
00037             _alive = false;
00038         }
00039     }
```

8.5.3.5 damageHP() [1/2]

```
void Health.damageHP (
    int damage) [inline]
```

Definition at line 97 of file [Health.cs](#).

```
00098     {
00099         _hp = Math.Max(_hp - damage, 0);
00100         _alive = Convert.ToBoolean(_hp);
00101     }
```

8.5.3.6 damageHP() [2/2]

```
void Health.damageHP (  
    int damage) [inline]
```

Definition at line 97 of file [Health.cs](#).

```
00098     {  
00099         _hp = Math.Max(_hp - damage, 0);  
00100         _alive = Convert.ToBoolean(_hp);  
00101     }
```

8.5.3.7 damageShield() [1/2]

```
void Health.damageShield (  
    int damage) [inline]
```

Definition at line 92 of file [Health.cs](#).

```
00093     {  
00094         _shield = Math.Max(_shield - damage, 0);  
00095     }
```

8.5.3.8 damageShield() [2/2]

```
void Health.damageShield (  
    int damage) [inline]
```

Definition at line 92 of file [Health.cs](#).

```
00093     {  
00094         _shield = Math.Max(_shield - damage, 0);  
00095     }
```

8.5.3.9 DestroyShield() [1/2]

```
void Health.DestroyShield () [inline]
```

Definition at line 78 of file [Health.cs](#).

```
00079     {  
00080         _shield = 0;  
00081     }
```

8.5.3.10 DestroyShield() [2/2]

```
void Health.DestroyShield () [inline]
```

Definition at line 78 of file [Health.cs](#).

```
00079     {  
00080         _shield = 0;  
00081     }
```

8.5.3.11 HealBoth() [1/2]

```
void Health.HealBoth (
    int healAmount) [inline]
```

Definition at line 66 of file [Health.cs](#).

```
00067     {
00068         int healthMissing = _hpMax - _hp;
00069
00070         _hp = Math.Min(_hp + healAmount, _hpMax);
00071
00072         if (healAmount > healthMissing)
00073         {
00074             _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00075         }
00076     }
```

8.5.3.12 HealBoth() [2/2]

```
void Health.HealBoth (
    int healAmount) [inline]
```

Definition at line 66 of file [Health.cs](#).

```
00067     {
00068         int healthMissing = _hpMax - _hp;
00069
00070         _hp = Math.Min(_hp + healAmount, _hpMax);
00071
00072         if (healAmount > healthMissing)
00073         {
00074             _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00075         }
00076     }
```

8.5.3.13 HealHP() [1/2]

```
void Health.HealHP (
    int amount) [inline]
```

Definition at line 56 of file [Health.cs](#).

```
00057     {
00058         _hp = Math.Min(_hp + amount, _hpMax);
00059     }
```

8.5.3.14 HealHP() [2/2]

```
void Health.HealHP (
    int amount) [inline]
```

Definition at line 56 of file [Health.cs](#).

```
00057     {
00058         _hp = Math.Min(_hp + amount, _hpMax);
00059     }
```

8.5.3.15 HealShield() [1/2]

```
void Health.HealShield (
    int amount) [inline]
```

Definition at line 61 of file [Health.cs](#).

```
00062     {
00063         _shield = Math.Min(_shield + amount, _shieldMax);
00064     }
```

8.5.3.16 HealShield() [2/2]

```
void Health.HealShield (  
    int amount) [inline]
```

Definition at line 61 of file [Health.cs](#).

```
00062     {  
00063         _shield = Math.Min(_shield + amount, _shieldMax);  
00064     }
```

8.5.3.17 RestoreHealth() [1/2]

```
void Health.RestoreHealth () [inline]
```

Definition at line 82 of file [Health.cs](#).

```
00083     {  
00084         _hp = _hpMax;  
00085     }
```

8.5.3.18 RestoreHealth() [2/2]

```
void Health.RestoreHealth () [inline]
```

Definition at line 82 of file [Health.cs](#).

```
00083     {  
00084         _hp = _hpMax;  
00085     }
```

8.5.3.19 RestoreShield() [1/2]

```
void Health.RestoreShield () [inline]
```

Definition at line 87 of file [Health.cs](#).

```
00088     {  
00089         _shield = _shieldMax;  
00090     }
```

8.5.3.20 RestoreShield() [2/2]

```
void Health.RestoreShield () [inline]
```

Definition at line 87 of file [Health.cs](#).

```
00088     {  
00089         _shield = _shieldMax;  
00090     }
```

8.5.3.21 Revive() [1/2]

```
void Health.Revive () [inline]
```

Definition at line 103 of file [Health.cs](#).

```
00104     {  
00105         _hp = 1;  
00106         _shield = 0;  
00107         _alive = true;  
00108     }
```

8.5.3.22 Revive() [2/2]

```
void Health.Revive () [inline]
```

Definition at line 103 of file [Health.cs](#).

```
00104     {  
00105         _hp = 1;  
00106         _shield = 0;  
00107         _alive = true;  
00108     }
```

8.5.4 Member Data Documentation

8.5.4.1 _damageBias

```
const int Health._damageBias = 12 [static]
```

Definition at line 14 of file [Health.cs](#).

8.5.5 Property Documentation

8.5.5.1 _alive

```
bool Health._alive = true [get]
```

Definition at line 13 of file [Health.cs](#).

```
00013 { get; private set; } = true;
```

8.5.5.2 _hp

```
int Health._hp [get]
```

Definition at line 5 of file [Health.cs](#).

```
00005 { get; private set; }
```

8.5.5.3 _hpMax

```
int Health._hpMax [get]
```

Definition at line 6 of file [Health.cs](#).

```
00006 { get; private set; }
```

8.5.5.4 _resistanceElectric

```
int Health._resistanceElectric [get]
```

Definition at line 10 of file [Health.cs](#).

```
00010 { get; private set; }
```

8.5.5.5 `_resistancePhysical`

```
int Health._resistancePhysical [get]
```

Definition at line 9 of file [Health.cs](#).

```
00009 { get; private set; }
```

8.5.5.6 `_shield`

```
int Health._shield [get]
```

Definition at line 7 of file [Health.cs](#).

```
00007 { get; private set; }
```

8.5.5.7 `_shieldMax`

```
int Health._shieldMax [get]
```

Definition at line 8 of file [Health.cs](#).

```
00008 { get; private set; }
```

8.5.5.8 `_vulnElectric`

```
int Health._vulnElectric [get]
```

Definition at line 12 of file [Health.cs](#).

```
00012 { get; private set; }
```

8.5.5.9 `_vulnPhysical`

```
int Health._vulnPhysical [get]
```

Definition at line 11 of file [Health.cs](#).

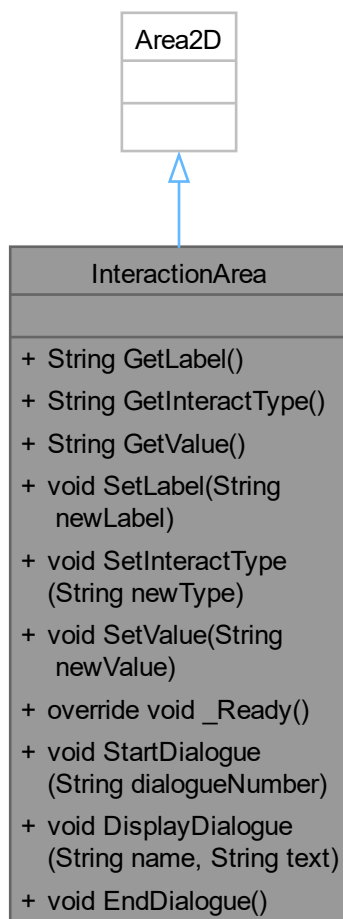
```
00011 { get; private set; }
```

The documentation for this class was generated from the following files:

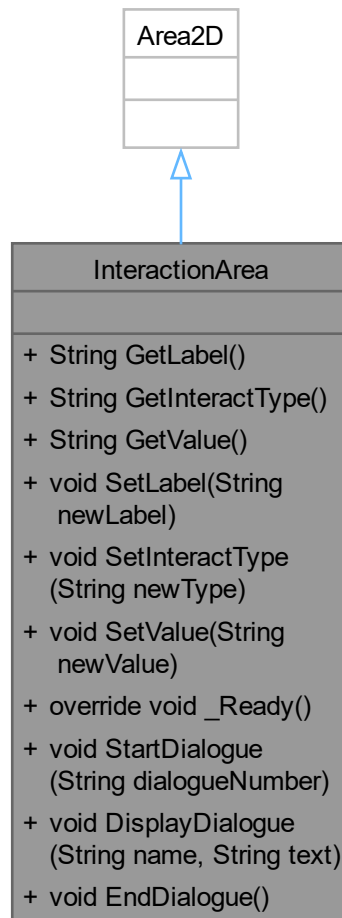
- `game-systems/TestProject1/Health.cs`
- `systems/TestProject1/Health.cs`

8.6 InteractionArea Class Reference

Inheritance diagram for InteractionArea:



Collaboration diagram for InteractionArea:



Public Member Functions

- String [GetLabel](#) ()
- String [GetInteractType](#) ()
- String [GetValue](#) ()
- void [SetLabel](#) (String newLabel)
- void [SetInteractType](#) (String newType)
- void [SetValue](#) (String newValue)
- override void [_Ready](#) ()
- void [StartDialogue](#) (String dialogueNumber)
- void [DisplayDialogue](#) (String name, String text)
- void [EndDialogue](#) ()

8.6.1 Detailed Description

Definition at line 4 of file [InteractionArea.cs](#).

8.6.2 Member Function Documentation

8.6.2.1 _Ready()

```
override void InteractionArea._Ready () [inline]
```

Definition at line 32 of file [InteractionArea.cs](#).

```
00032     {  
00033         GetNode<NinePatchRect>("DialogueManager").Visible = false;  
00034     }
```

8.6.2.2 DisplayDialogue()

```
void InteractionArea.DisplayDialogue (  
    String name,  
    String text) [inline]
```

Definition at line 60 of file [InteractionArea.cs](#).

```
00060     {  
00061         GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Name").Text = name;  
00062         GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Text").Text = text;  
00063     }
```

8.6.2.3 EndDialogue()

```
void InteractionArea.EndDialogue () [inline]
```

Definition at line 65 of file [InteractionArea.cs](#).

```
00065     {  
00066         GetNode<NinePatchRect>("DialogueManager").Visible = false;  
00067         QueueFree();  
00068     }
```

8.6.2.4 GetInteractType()

```
String InteractionArea.GetInteractType () [inline]
```

Definition at line 16 of file [InteractionArea.cs](#).

```
00016     {  
00017         return interactionType;  
00018     }
```

8.6.2.5 GetLabel()

```
String InteractionArea.GetLabel () [inline]
```

Definition at line 13 of file [InteractionArea.cs](#).

```
00013     {  
00014         return interactionLabel;  
00015     }
```

8.6.2.6 GetValue()

```
String InteractionArea.GetValue () [inline]
```

Definition at line 19 of file [InteractionArea.cs](#).

```
00019      {
00020          return interactionValue;
00021      }
```

8.6.2.7 SetInteractType()

```
void InteractionArea.SetInteractType (
    String newType) [inline]
```

Definition at line 25 of file [InteractionArea.cs](#).

```
00025      {
00026          interactionType = newType;
00027      }
```

8.6.2.8 SetLabel()

```
void InteractionArea.SetLabel (
    String newLabel) [inline]
```

Definition at line 22 of file [InteractionArea.cs](#).

```
00022      {
00023          interactionLabel = newLabel;
00024      }
```

8.6.2.9 SetValue()

```
void InteractionArea.SetValue (
    String newValue) [inline]
```

Definition at line 28 of file [InteractionArea.cs](#).

```
00028      {
00029          interactionValue = newValue;
00030      }
```

8.6.2.10 StartDialogue()

```
void InteractionArea.StartDialogue (
    String dialogueNumber) [inline]
```

Definition at line 37 of file [InteractionArea.cs](#).

```
00037      {
00038          GetNode<NinePatchRect>("DialogueManager").Visible = true;
00039          switch(dialogueNumber){
00040              case "0":
00041                  DisplayDialogue("Info!", "Welcome to Heidelberg Dynamics! You are L4lk4, and need to
00042                  escape this facility." + " Use [W][A][S][D] to move to the next room. Press [E] to
00043                  continue.");
00044                  break;
00045              case "1":
00046                  DisplayDialogue("Info!", "You need to be careful. This facility is full of enemy
00047                  turrets that will" + " shoot you on sight. Use [LMB] to shoot your weapon.");
00048                  break;
00049              case "2":
```

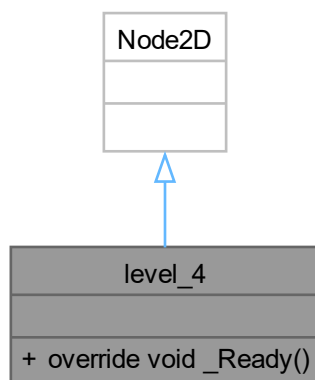
```
00049         DisplayDialogue("Info!", "You need to collect a keycard to open the door to the next
00050         level.");
00051         break;
00052     case "3":
00053         DisplayDialogue("Info!", "You escaped! You can now enjoy your freedom.");
00054         break;
00055     default:
00056         EndDialogue();
00057         break;
00058     }
```

The documentation for this class was generated from the following file:

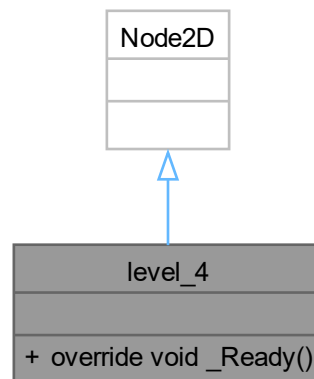
- InteractionArea.cs

8.7 level_4 Class Reference

Inheritance diagram for level_4:



Collaboration diagram for level_4:



Public Member Functions

- `override void \_Ready ()`

8.7.1 Detailed Description

Definition at line 4 of file [level_4.cs](#).

8.7.2 Member Function Documentation

8.7.2.1 `_Ready()`

```
override void level_4._Ready () [inline]
```

Definition at line 6 of file [level_4.cs](#).

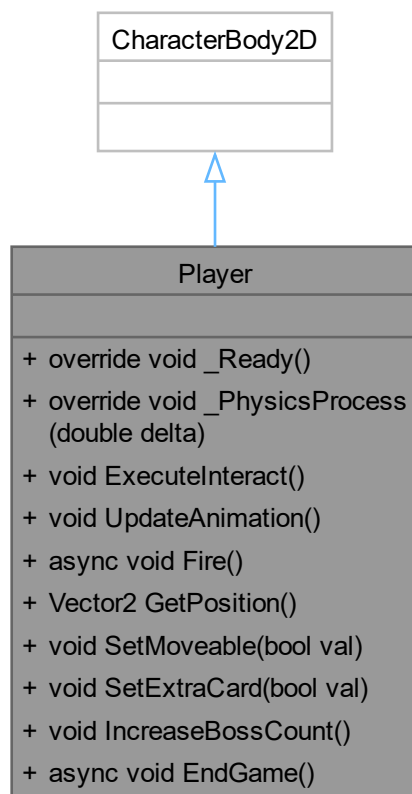
```
00006      {
00007          GetNode<Player> ("Player").SetExtraCard(false);
00008          GetNode<StaticBody2D> ("keycard2").GetNode<InteractionArea> ("InteractionArea").SetValue("keycard2");
00009      }
```

The documentation for this class was generated from the following file:

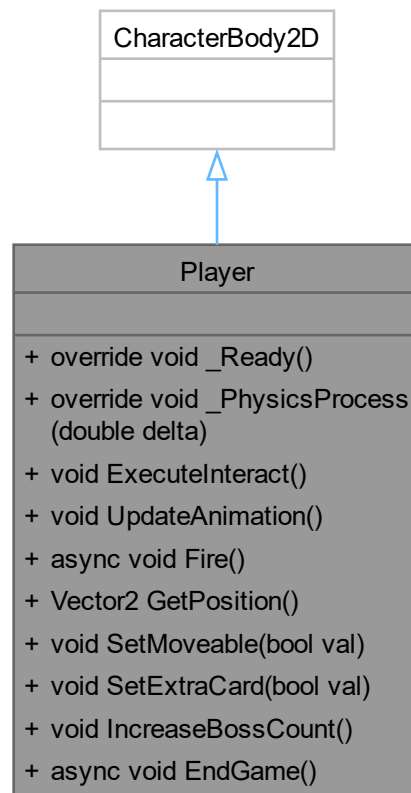
- `levels/level_4.cs`

8.8 Player Class Reference

Inheritance diagram for Player:



Collaboration diagram for Player:



Public Member Functions

- override void `_Ready` ()
- override void `_PhysicsProcess` (double delta)
- void `ExecuteInteract` ()
- void `UpdateAnimation` ()
- async void `Fire` ()
- Vector2 `GetPosition` ()
- void `SetMoveable` (bool val)
- void `SetExtraCard` (bool val)
- void `IncreaseBossCount` ()
- async void `EndGame` ()

8.8.1 Detailed Description

Definition at line 4 of file [Player.cs](#).

8.8.2 Member Function Documentation

8.8.2.1 _PhysicsProcess()

```
override void Player._PhysicsProcess (
    double delta) [inline]
```

Definition at line 25 of file [Player.cs](#).

```
00025         {
00026             GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Health").Value =
GetNode<PlayerHealth>("Health").health._hp;
00027             GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Shield").Value =
GetNode<PlayerHealth>("Health").health._shield;
00028             if(!GetNode<PlayerHealth>("Health").health._alive){
00029                 moveable = false;
00030                 GetParent().GetNode<CanvasLayer>("HUD").GetNode<NinePatchRect>("Defeat").Visible = true;
00031             }
00032             //get input directions
00033             Vector2 inputDirection = new Vector2(
00034                 Input.GetActionRawStrength("right") - Input.GetActionRawStrength("left"),
00035                 Input.GetActionRawStrength("down") - Input.GetActionRawStrength("up")
00036             );
00037             if(moveable){
00038                 LookAt(GetGlobalMousePosition());
00039
00040                 //fire bullet if pressed
00041                 if(Input.IsActionPressed("fire") && canFire){
00042                     Fire();
00043                 }
00044                 //update velocity and move character
00045                 this.Velocity = inputDirection * moveSpeed;
00046                 MoveAndSlide();
00047
00048                 UpdateAnimation();
00049             }
00050             //execute interaction if needed
00051             if(Input.IsActionPressed("interact")){
00052                 if(GetNode<PlayerHealth>("Health").health._alive)
00053                     ExecuteInteract();
00054                 else{
00055                     GetTree().ChangeSceneToFile("res://levels/level_1.tscn");
00056                 }
00057             }
00058
00059             if(Input.IsActionPressed("exit")){
00060                 GetTree().Quit();
00061             }
00062
00063             if(Input.IsActionPressed("cheat")){
00064                 GetTree().ChangeSceneToFile("res://levels/level_6.tscn");
00065             }
00066             if(Input.IsActionPressed("speed")){
00067                 moveSpeed = 500;
00068             }
00069
00070             if(bossCount == 2){
00071                 EndGame();
00072             }
00073         }
```

8.8.2.2 _Ready()

```
override void Player._Ready () [inline]
```

Definition at line 18 of file [Player.cs](#).

```
00018         {
00019             moveable = true;
00020             this.gotKeycard = false;
00021             animationTree = GetNode<AnimationTree>("AnimationTree");
00022             stateMachine = (AnimationNodeStateMachinePlayback)animationTree.Get("parameters/playback");
00023         }
```

8.8.2.3 EndGame()

```
async void Player.EndGame () [inline]
```

Definition at line 180 of file [Player.cs](#).

```
00180      {
00181          await ToSignal(GetTree().CreateTimer(10), "timeout");
00182          GetTree().ChangeSceneToFile("res://levels/level_7.tscn");
00183      }
```

8.8.2.4 ExecuteInteract()

```
void Player.ExecuteInteract () [inline]
```

Definition at line 75 of file [Player.cs](#).

```
00075      {
00076          if(currentInteract!=null){
00077              //check which type of interaction is executed
00078              switch(currentInteract.GetInteractType()){
00079                  case "collect":
00080                      //check if a keycard or a weapon is picked up
00081                      if(currentInteract.GetValue()=="keycard"){
00082                          this.gotKeycard = true;
00083                          GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D").Visible
00084                          = true;
00085                          //update interaction labels and visibility of keycard
00086                          GetParent().GetNode<StaticBody2D>("keycard").Visible = false;
00087                          GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00088                          GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetLabel("");
00089                          //change label of door
00090                          if(gotExtraCard){
00091                              GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
00092                              next_level");
00093                          }
00094                      }
00095                      else if(currentInteract.GetValue()=="keycard2"){
00096                          this.gotExtraCard = true;
00097                          GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D2").Visible
00098                          = true;
00099                          //update interaction labels and visibility of keycard
00100                          GetParent().GetNode<StaticBody2D>("keycard2").Visible = false;
00101                          GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00102                          GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetLabel("");
00103                          //change label of door
00104                          if(gotKeycard){
00105                              GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
00106                              next_level");
00107                          }
00108                      }
00109                      break;
00110                  case "next_level":
00111                      if(gotKeycard && gotExtraCard){
00112                          GetTree().ChangeSceneToFile("res://levels/" +
00113                          GetParent().GetNode<InteractionArea>("NextLevel").GetValue());
00114                      }
00115                      break;
00116                  case "dialogue":
00117                      moveable = true;
00118                      currentInteract.EndDialogue();
00119                      break;
00120              }
00121          }
00122      }
```

8.8.2.5 Fire()

```
async void Player.Fire () [inline]
```

Definition at line 152 of file [Player.cs](#).

```
00152         {
00153             GetNode<AudioStreamPlayer>("BulletSound").Play();
00154             RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00155             GetParent().AddChild(bulletInstance);
00156             bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00157             bulletInstance.RotationDegrees = RotationDegrees;
00158             bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00159             canFire = false;
00160             await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00161             canFire = true;
00162         }
```

8.8.2.6 GetPosition()

```
Vector2 Player.GetPosition () [inline]
```

Definition at line 164 of file [Player.cs](#).

```
00164         {
00165             return Position;
00166         }
```

8.8.2.7 IncreaseBossCount()

```
void Player.IncreaseBossCount () [inline]
```

Definition at line 176 of file [Player.cs](#).

```
00176         {
00177             bossCount++;
00178         }
```

8.8.2.8 SetExtraCard()

```
void Player.SetExtraCard (
    bool val) [inline]
```

Definition at line 172 of file [Player.cs](#).

```
00172         {
00173             gotExtraCard = val;
00174         }
```

8.8.2.9 SetMoveable()

```
void Player.SetMoveable (
    bool val) [inline]
```

Definition at line 168 of file [Player.cs](#).

```
00168         {
00169             moveable = val;
00170         }
```

8.8.2.10 UpdateAnimation()

```
void Player.UpdateAnimation () [inline]
```

Definition at line 121 of file [Player.cs](#).

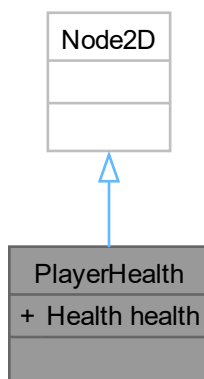
```
00121         {  
00122             if(this.Velocity!=Vector2.Zero){  
00123                 stateMachine.Travel("walk");  
00124             }  
00125             else{  
00126                 stateMachine.Travel("idle");  
00127             }  
00128         }
```

The documentation for this class was generated from the following file:

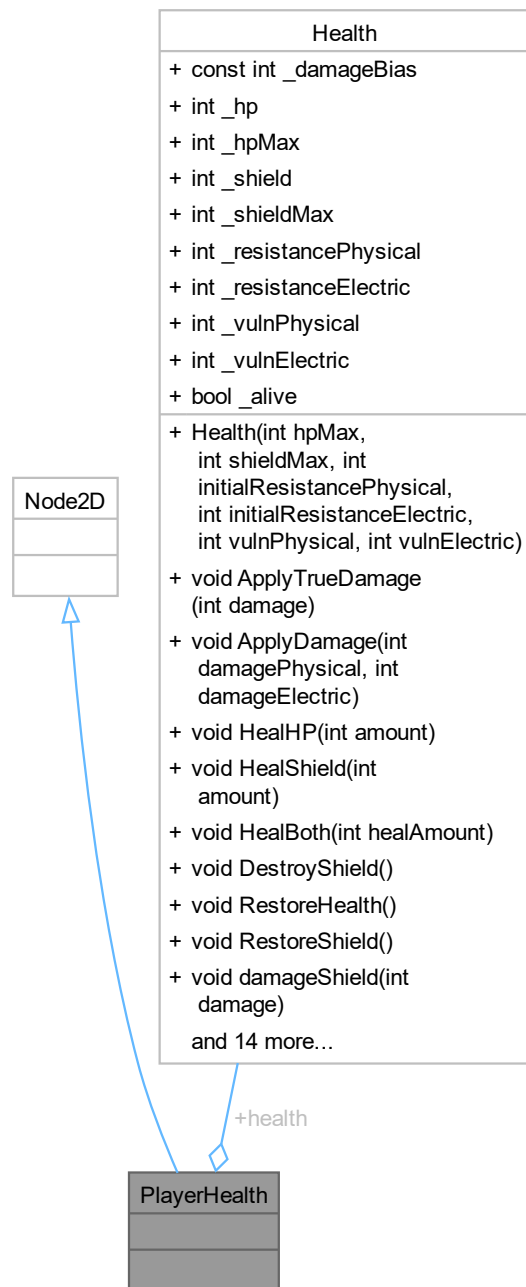
- characters/Player.cs

8.9 PlayerHealth Class Reference

Inheritance diagram for PlayerHealth:



Collaboration diagram for PlayerHealth:



Public Attributes

- `Health health = new Health(1000,1000,100,100,100,100)`

8.9.1 Detailed Description

Definition at line 4 of file [PlayerHealth.cs](#).

8.9.2 Member Data Documentation

8.9.2.1 health

```
Health PlayerHealth.health = new Health(1000,1000,100,100,100,100)
```

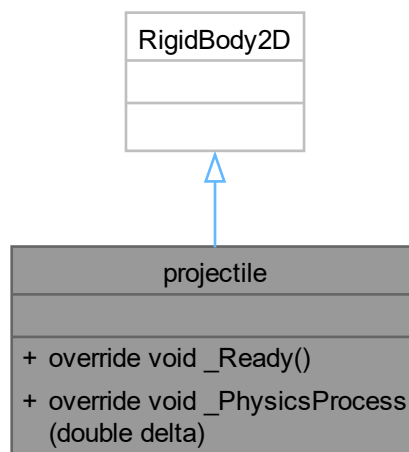
Definition at line 6 of file [PlayerHealth.cs](#).

The documentation for this class was generated from the following file:

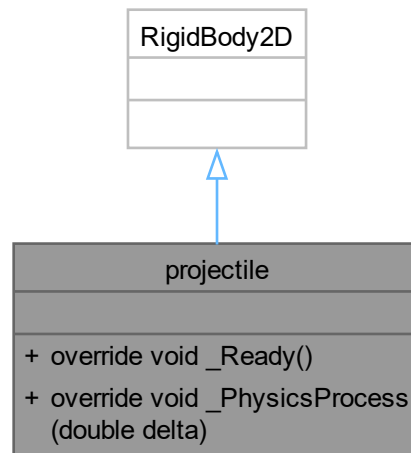
- [characters/PlayerHealth.cs](#)

8.10 projectile Class Reference

Inheritance diagram for projectile:



Collaboration diagram for projectile:



Public Member Functions

- `override void \_Ready ()`
- `override void \_PhysicsProcess (double delta)`

8.10.1 Detailed Description

Definition at line 4 of file [projectile.cs](#).

8.10.2 Member Function Documentation

8.10.2.1 `_PhysicsProcess()`

```
override void projectile._PhysicsProcess (
    double delta) [inline]
```

Definition at line 38 of file [projectile.cs](#).

```
00038                                     {
00039         if (timer.IsStopped())
00040             QueueFree();
00041     }
```

8.10.2.2 `_Ready()`

```
override void projectile._Ready () [inline]
```

Definition at line 9 of file [projectile.cs](#).

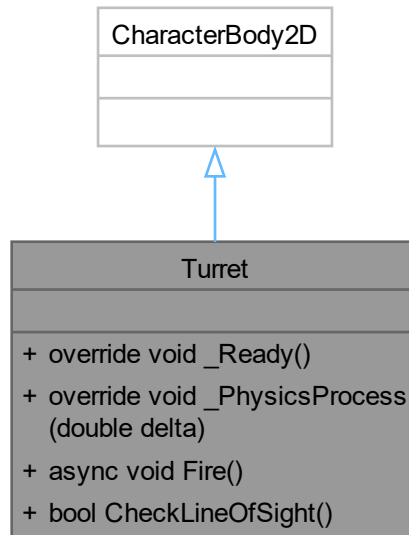
```
00009                                     {
00010         timer = new Timer();
00011         this.AddChild(timer);
00012         timer.WaitTime = 0.34;
00013         timer.OneShot = true;
00014         timer.Start();
00015     }
```

The documentation for this class was generated from the following file:

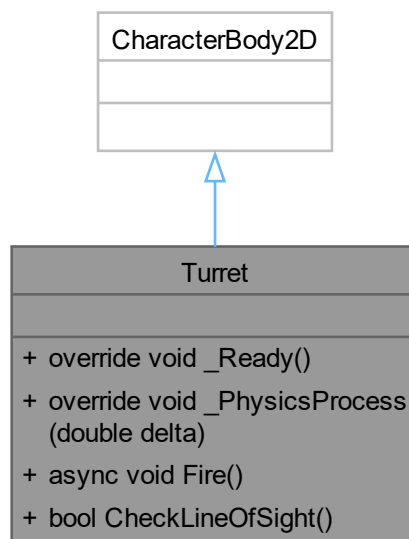
- `projectiles/projectile.cs`

8.11 Turret Class Reference

Inheritance diagram for Turret:



Collaboration diagram for Turret:



Public Member Functions

- override void [_Ready](#) ()
- override void [_PhysicsProcess](#) (double delta)
- async void [Fire](#) ()
- bool [CheckLineOfSight](#) ()

8.11.1 Detailed Description

Definition at line 4 of file [Turret.cs](#).

8.11.2 Member Function Documentation

8.11.2.1 [_PhysicsProcess\(\)](#)

```
override void Turret._PhysicsProcess (
    double delta) [inline]
```

Definition at line 15 of file [Turret.cs](#).

```
00015         {
00016             if(!GetNode<TurretHealth>("Health").health._alive){
00017                 hasWeapon = false;
00018                 GetNode<Sprite2D>("Turret").Visible = false;
00019             }
00020             seesPlayer = CheckLineOfSight();
00021             if(hasWeapon){
00022                 LookAt(GetParent().GetNode<Player>("Player").GetPosition());
00023                 if(seesPlayer && canFire){
00024                     Fire();
00025                 }
00026             }
00027         }
```

8.11.2.2 [_Ready\(\)](#)

```
override void Turret._Ready () [inline]
```

Definition at line 12 of file [Turret.cs](#).

```
00012         {
00013     }
```

8.11.2.3 [CheckLineOfSight\(\)](#)

```
bool Turret.CheckLineOfSight () [inline]
```

Definition at line 41 of file [Turret.cs](#).

```
00041         {
00042             Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00043             double distance = Mathf.Sqrt((playerPos.X - Position.X)*(playerPos.X - Position.X) +
00044                 (playerPos.Y - Position.Y)*(playerPos.Y - Position.Y));
00045             if(distance < 300){
00046                 return true;
00047             }
00048             return false;
00049         }
```

8.11.2.4 Fire()

```
async void Turret.Fire () [inline]
```

Definition at line 29 of file [Turret.cs](#).

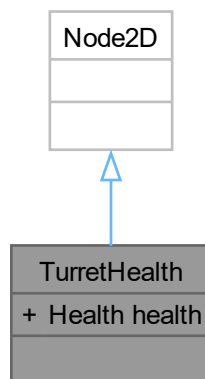
```
00029 {
00030     GetNode<AudioStreamPlayer>("BulletSound").Play();
00031     RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00032     GetParent().AddChild(bulletInstance);
00033     bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00034     bulletInstance.RotationDegrees = RotationDegrees;
00035     bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation), new Vector2());
00036     canFire = false;
00037     await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00038     canFire = true;
00039 }
```

The documentation for this class was generated from the following file:

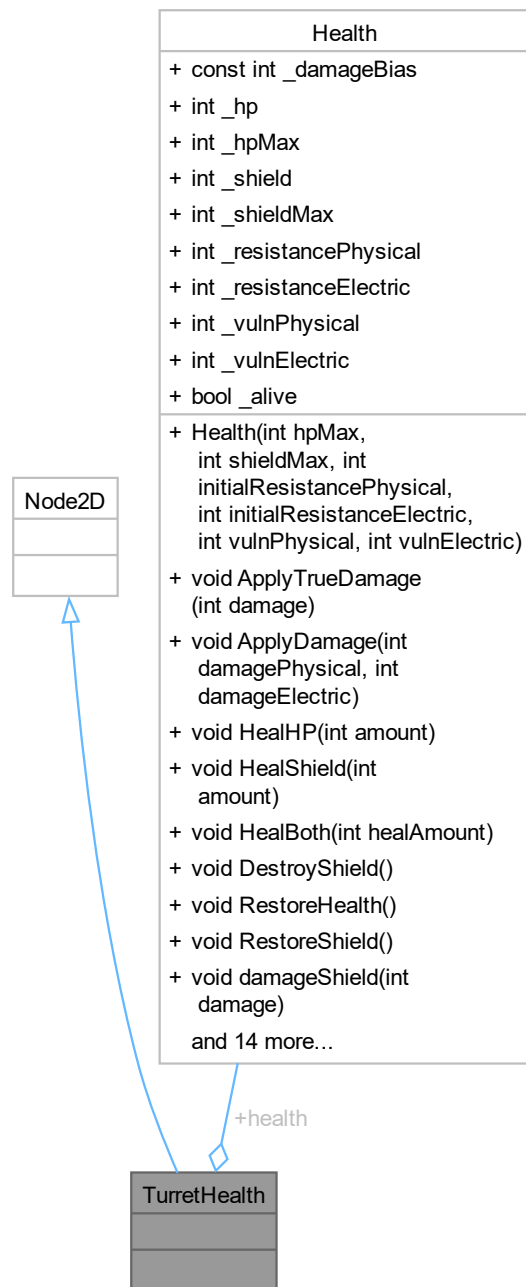
- characters/Turret.cs

8.12 TurretHealth Class Reference

Inheritance diagram for TurretHealth:



Collaboration diagram for TurretHealth:



Public Attributes

- `Health health = new Health(100,100,100,100,100,100)`

8.12.1 Detailed Description

Definition at line 4 of file [TurretHealth.cs](#).

8.12.2 Member Data Documentation

8.12.2.1 health

```
Health TurretHealth.health = new Health(100,100,100,100,100,100)
```

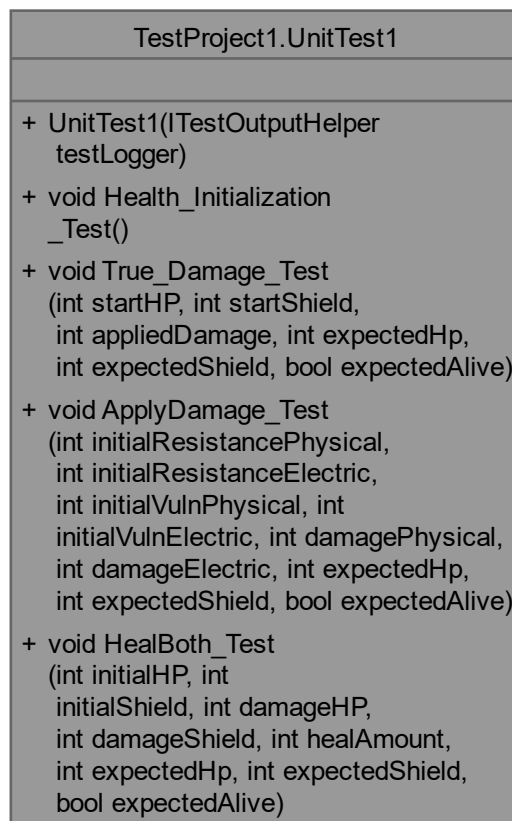
Definition at line 6 of file [TurretHealth.cs](#).

The documentation for this class was generated from the following file:

- [characters/TurretHealth.cs](#)

8.13 TestProject1.UnitTest1 Class Reference

Collaboration diagram for TestProject1.UnitTest1:



Public Member Functions

- [UnitTest1](#) (ITestOutputHelper testLogger)
- void [Health_Initialization_Test](#) ()
- void [True_Damage_Test](#) (int startHP, int startShield, int appliedDamage, int expectedHp, int expectedShield, bool expectedAlive)
- void [ApplyDamage_Test](#) (int initialResistancePhysical, int initialResistanceElectric, int initialVulnPhysical, int initialVulnElectric, int damagePhysical, int damageElectric, int expectedHp, int expectedShield, bool expectedAlive)
- void [HealBoth_Test](#) (int initialHP, int initialShield, int damageHP, int damageShield, int healAmount, int expectedHp, int expectedShield, bool expectedAlive)

8.13.1 Detailed Description

Definition at line 5 of file [UnitTest1.cs](#).

8.13.2 Constructor & Destructor Documentation

8.13.2.1 UnitTest1()

```
TestProject1.UnitTest1.UnitTest1 (
    ITestOutputHelper testLogger) [inline]
```

Definition at line 8 of file [UnitTest1.cs](#).

```
00009     {
00010         this._testLogger = testLogger;
00011     }
```

8.13.3 Member Function Documentation

8.13.3.1 ApplyDamage_Test()

```
void TestProject1.UnitTest1.ApplyDamage_Test (
    int initialResistancePhysical,
    int initialResistanceElectric,
    int initialVulnPhysical,
    int initialVulnElectric,
    int damagePhysical,
    int damageElectric,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Definition at line 64 of file [UnitTest1.cs](#).

```
00069     {
00070         // Arrange
00071         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00072             initialVulnPhysical, initialVulnElectric);
00073
00074         // Act
00075         health.ApplyDamage(damagePhysical, damageElectric);
00076
00077         // Assert
00078         Assert.Equal(expectedHp, health._hp);
00079         Assert.Equal(expectedShield, health._shield);
00080         Assert.Equal(expectedAlive, health._alive);
00081     }
```

8.13.3.2 HealBoth_Test()

```
void TestProject1.UnitTest1.HealBoth_Test (
    int initialHP,
    int initialShield,
    int damageHP,
    int damageShield,
    int healAmount,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Definition at line 92 of file [UnitTest1.cs](#).

```
00096     {
00097         // Arrange
00098         Health health = new Health(initialHP, initialShield, 0, 0, 0, 0);
00099
00100         // Act
00101         health.damageHP(damageHP);
00102         health.damageShield(damageShield);
00103
00104         health.HealBoth(healAmount);
00105
00106         // Assert
00107         Assert.Equal(expectedHp, health._hp);
00108         Assert.Equal(expectedShield, health._shield);
00109         Assert.Equal(expectedAlive, health._alive);
00110     }
```

8.13.3.3 Health_Initialization_Test()

```
void TestProject1.UnitTest1.Health_Initialization_Test () [inline]
```

Definition at line 14 of file [UnitTest1.cs](#).

```
00015     {
00016         int initialResistancePhysical = 100;
00017         int initialResistanceElectric = 300;
00018         int initialVulnPhysical = 0;
00019         int initialVulnElectric = 200;
00020
00021         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00022             initialVulnPhysical, initialVulnElectric);
00023
00024         Assert.Equal(1000, health._hpMax);
00025         Assert.Equal(1000, health._hp);
00026         Assert.Equal(1000, health._shield);
00027         Assert.Equal(initialResistancePhysical, health._resistancePhysical);
00028         Assert.Equal(initialResistanceElectric, health._resistanceElectric);
00029         Assert.Equal(initialVulnPhysical, health._vulnPhysical);
00030         Assert.Equal(initialVulnElectric, health._vulnElectric);
00031         Assert.True(health._alive);
00032     }
```

8.13.3.4 True_Damage_Test()

```
void TestProject1.UnitTest1.True_Damage_Test (
    int startHP,
    int startShield,
    int appliedDamage,
    int expectedHp,
    int expectedShield,
    bool expectedAlive) [inline]
```

Definition at line 41 of file [UnitTest1.cs](#).

```
00044     {
00045         // Arrange
```

```
00046         Health health = new(startHP, startShield, 1000, 1000, 0, 0);
00047
00048         // Act
00049         health.ApplyTrueDamage(appliedDamage);
00050
00051         // Assert
00052         Assert.Equal(expectedHp, health._hp);
00053         Assert.Equal(expectedShield, health._shield);
00054         Assert.Equal(expectedAlive, health._alive);
00055     }
```

The documentation for this class was generated from the following file:

- game-systems/TestProject1/UnitTest1.cs

Chapter 9

File Documentation

9.1 BossHealth.cs

```
00001 using Godot;
00002 using System;
00003
00004 public partial class BossHealth : Node2D
00005 {
00006     public Health health = new Health(1000,1000,100,100,100,100);
00007 }
```

9.2 Bossturret.cs

```
00001 using Godot;
00002 using System;
00003
00004 public partial class Bossturret : CharacterBody2D
00005 {
00006     private bool hasWeapon = true;
00007     private bool counter = false;
00008     private double firerate = 0.7;
00009     private bool seesPlayer=false;
00010     private PackedScene bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn");
00011     private bool canFire = true;
00012
00013     public override void _PhysicsProcess(double delta){
00014         if(!GetNode<BossHealth>("Health").health._alive){
00015             hasWeapon = false;
00016             if(!counter){
00017                 GetParent().GetNode<Player>("Player").IncreaseBossCount();
00018                 counter = true;
00019             }
00020             GetNode<Sprite2D>("Turret").Visible = false;
00021         }
00022         seesPlayer = CheckLineOfSight();
00023         if(hasWeapon){
00024             LookAt(GetParent().GetNode<Player>("Player").GetPosition());
00025             if(seesPlayer && canFire){
00026                 Fire1();
00027                 Fire2();
00028             }
00029         }
00030     }
00031
00032     public async void Fire1(){
00033         GetNode<AudioStreamPlayer>("BulletSound").Play();
00034         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00035         GetParent().AddChild(bulletInstance);
00036         bulletInstance.Position = GetNode<Node2D>("BulletPoint1").GlobalPosition;
00037         bulletInstance.RotationDegrees = RotationDegrees;
00038         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation),new Vector2());
00039         canFire = false;
00040         await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00041         canFire = true;
00042     }
00043     public async void Fire2(){
00044         await ToSignal(GetTree().CreateTimer(0.35), "timeout");
00045     }
00046 }
```

```

00045         GetNode<AudioStreamPlayer>("BulletSound").Play();
00046         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00047         GetParent().AddChild(bulletInstance);
00048         bulletInstance.Position = GetNode<Node2D>("BulletPoint2").GlobalPosition;
00049         bulletInstance.RotationDegrees = RotationDegrees;
00050         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation),new Vector2());
00051     }
00052
00053     public bool CheckLineOfSight(){
00054         Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00055         double distance = Mathf.Sqrt((playerPos.X - Position.X)*(playerPos.X - Position.X) +
00056             (playerPos.Y - Position.Y)*(playerPos.Y - Position.Y));
00057         if(distance < 300){
00058             return true;
00059         }
00060         return false;
00061     }
00062 }

```

9.3 NinePatchRect.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class DialogueManager : Node2D
00005 {
00006
00007     public override void _Ready(){
00008         Visible = true;
00009     }
00010
00011
00012     public void StartDialogue(String dialogueNumber){
00013         //GetParent().SetMoveable(false);
00014         Visible = false;
00015         switch(dialogueNumber){
00016             case "0":
00017                 DisplayDialogue("Game","Welcome to Heidelberg Dynamics! You are L4lk4, and need to
escape this facility."
00018                                     + " Use [W][A][S][D] to move to the next room. Press [E] to
continue");
00019                 break;
00020             case "1":
00021                 DisplayDialogue("Game","You need to be careful. This facility is full of enemie
turrets that will" +
00022                                     " shoot you on sight. Use [LMB] to shoot your weapon");
00023                 break;
00024             default:
00025                 EndDialogue();
00026                 break;
00027         }
00028     }
00029
00030     public void DisplayDialogue(String name,String text){
00031         GetNode<RichTextLabel>("Name").Text = name;
00032         GetNode<RichTextLabel>("Text").Text = text;
00033     }
00034
00035     public void EndDialogue(){
00036         Visible = false;
00037         //GetParent().SetMoveable(true);
00038     }
00039
00040     private void _Input(){
00041         if(Input.IsActionPressed("continue")){
00042             EndDialogue();
00043         }
00044     }
00045 }
00046 }

```

9.4 Player.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class Player : CharacterBody2D
00005 {
00006     private float moveSpeed = 175;

```

```

00007     private bool gotKeycard = false;
00008     private bool gotExtraCard = true;
00009     private bool moveable = true;
00010     private int bossCount = 0;
00011     private InteractionArea currentInteract;
00012     private AnimationTree animationTree;
00013     private AnimationNodeStateMachinePlayback stateMachine;
00014     private double firerate = 0.5;
00015     private PackedScene bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn");
00016     private bool canFire = true;
00017
00018     public override void _Ready(){
00019         moveable = true;
00020         this.gotKeycard = false;
00021         animationTree = GetNode<AnimationTree>("AnimationTree");
00022         stateMachine = (AnimationNodeStateMachinePlayback)animationTree.Get("parameters/playback");
00023     }
00024
00025     public override void _PhysicsProcess(double delta){
00026         GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Health").Value =
00027         GetNode<PlayerHealth>("Health").health._hp;
00028         GetParent().GetNode<CanvasLayer>("HUD").GetNode<ProgressBar>("Shield").Value =
00029         GetNode<PlayerHealth>("Health").health._shield;
00030         if(!GetNode<PlayerHealth>("Health").health._alive){
00031             moveable = false;
00032             GetParent().GetNode<CanvasLayer>("HUD").GetNode<NinePatchRect>("Defeat").Visible = true;
00033         }
00034         //get input directions
00035         Vector2 inputDirection = new Vector2(
00036             Input.GetActionRawStrength("right") - Input.GetActionRawStrength("left"),
00037             Input.GetActionRawStrength("down") - Input.GetActionRawStrength("up")
00038         );
00039         if(moveable){
00040             LookAt(GetGlobalMousePosition());
00041
00042             //fire bullet if pressed
00043             if(Input.IsActionPressed("fire") && canFire){
00044                 Fire();
00045             }
00046             //update velocity and move character
00047             this.Velocity = inputDirection * moveSpeed;
00048             MoveAndSlide();
00049
00050             UpdateAnimation();
00051         }
00052         //execute interaction if needed
00053         if(Input.IsActionPressed("interact")){
00054             if(GetNode<PlayerHealth>("Health").health._alive)
00055                 ExecuteInteract();
00056             else{
00057                 GetTree().ChangeSceneToFile("res://levels/level_1.tscn");
00058             }
00059         }
00060         if(Input.IsActionPressed("exit")){
00061             GetTree().Quit();
00062         }
00063         if(Input.IsActionPressed("cheat")){
00064             GetTree().ChangeSceneToFile("res://levels/level_6.tscn");
00065         }
00066         if(Input.IsActionPressed("speed")){
00067             moveSpeed = 500;
00068         }
00069
00070         if(bossCount == 2){
00071             EndGame();
00072         }
00073     }
00074
00075     public void ExecuteInteract(){
00076         if(currentInteract!=null){
00077             //check which type of interaction is executed
00078             switch(currentInteract.GetInteractType()){
00079                 case "collect":
00080                     //check if a keycard or a weapon is picked up
00081                     if(currentInteract.GetValue()=="keycard"){
00082                         this.gotKeycard = true;
00083                         GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D").Visible
00084                         = true;
00085
00086                         //update interaction labels and visibility of keycard
00087                         GetParent().GetNode<StaticBody2D>("keycard").Visible = false;
00088
00089                         GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00090
00091                         GetParent().GetNode<StaticBody2D>("keycard").GetNode<InteractionArea>("InteractionArea").SetLabel("");

```

```

00089             //change label of door
00090             if(gotExtraCard){
00091                 GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
next_level");
00092             }
00093         }
00094         else if(currentInteract.GetValue()=="keycard2"){
00095             this.gotExtraCard = true;
00096             GetParent().GetNode<CanvasLayer>("HUD").GetNode<Sprite2D>("Sprite2D2").Visible
= true;
00097             //update interaction labels and visibility of keycard
00098             GetParent().GetNode<StaticBody2D>("keycard2").Visible = false;
00099
00100             GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetInteractType("collected");
00101             GetParent().GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetLabel("");
00102             //change label of door
00103             if(gotKeycard){
00104                 GetParent().GetNode<InteractionArea>("NextLevel").SetLabel("[E] to enter
next_level");
00105             }
00106             break;
00107             case "next_level":
00108                 if(gotKeycard && gotExtraCard){
00109                     GetTree().ChangeSceneToFile("res://levels/" +
GetParent().GetNode<InteractionArea>("NextLevel").GetValue());
00110                 }
00111                 break;
00112             case "dialogue":
00113                 moveable = true;
00114                 currentInteract.EndDialogue();
00115                 break;
00116         }
00117     }
00118 }
00119
00120
00121 public void UpdateAnimation(){
00122     if(this.Velocity!=Vector2.Zero){
00123         stateMachine.Travel("walk");
00124     }
00125     else{
00126         stateMachine.Travel("idle");
00127     }
00128 }
00129
00130 private void _on_interaction_area_area_entered(InteractionArea area)
00131 {
00132     if(area.GetInteractType()!="heal"){
00133         this.currentInteract = area;
00134         this.currentInteract.GetNode<Label>("Label").Text = currentInteract.GetLabel();
00135         if(area.GetInteractType()=="dialogue"){
00136             area.StartDialogue(area.GetValue());
00137             moveable = false;
00138         }
00139     }
00140     else{
00141         GetNode<PlayerHealth>("Health").health.HealBoth(500);
00142         area.GetParent().QueueFree();
00143     }
00144 }
00145
00146 private void _on_interaction_area_area_exited(InteractionArea area)
00147 {
00148     area.GetNode<Label>("Label").Text = "";
00149     this.currentInteract = null;
00150 }
00151
00152 public async void Fire(){
00153     GetNode<AudioStreamPlayer>("BulletSound").Play();
00154     RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00155     GetParent().AddChild(bulletInstance);
00156     bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00157     bulletInstance.RotationDegrees = RotationDegrees;
00158     bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation),new Vector2());
00159     canFire = false;
00160     await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00161     canFire = true;
00162 }
00163
00164 public Vector2 GetPosition(){
00165     return Position;
00166 }
00167
00168 public void SetMoveable(bool val){
00169     moveable = val;

```

```

00170     }
00171
00172     public void SetExtraCard(bool val){
00173         gotExtraCard = val;
00174     }
00175
00176     public void IncreaseBossCount(){
00177         bossCount++;
00178     }
00179
00180     public async void EndGame(){
00181         await ToSignal(GetTree().CreateTimer(10), "timeout");
00182         GetTree().ChangeSceneToFile("res://levels/level_7.tscn");
00183     }
00184 }

```

9.5 PlayerHealth.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class PlayerHealth : Node2D
00005 {
00006     public Health health = new Health(1000,1000,100,100,100,100);
00007 }

```

9.6 Turret.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class Turret : CharacterBody2D
00005 {
00006     private bool hasWeapon = true;
00007     private double firerate = 0.65;
00008     private bool seesPlayer=false;
00009     private PackedScene bullet = GD.Load<PackedScene>("res://projectiles/projectile.tscn");
00010     private bool canFire = true;
00011
00012     public override void _Ready(){
00013     }
00014
00015     public override void _PhysicsProcess(double delta){
00016         if(!GetNode<TurretHealth>("Health").health._alive){
00017             hasWeapon = false;
00018             GetNode<Sprite2D>("Turret").Visible = false;
00019         }
00020         seesPlayer = CheckLineOfSight();
00021         if(hasWeapon){
00022             LookAt(GetParent().GetNode<Player>("Player").GetPosition());
00023             if(seesPlayer && canFire){
00024                 Fire();
00025             }
00026         }
00027     }
00028
00029     public async void Fire(){
00030         GetNode<AudioStreamPlayer>("BulletSound").Play();
00031         RigidBody2D bulletInstance = bullet.Instantiate<RigidBody2D>();
00032         GetParent().AddChild(bulletInstance);
00033         bulletInstance.Position = GetNode<Node2D>("BulletPoint").GlobalPosition;
00034         bulletInstance.RotationDegrees = RotationDegrees;
00035         bulletInstance.ApplyImpulse(new Vector2(750, 0).Rotated(Rotation),new Vector2());
00036         canFire = false;
00037         await ToSignal(GetTree().CreateTimer(firerate), "timeout");
00038         canFire = true;
00039     }
00040
00041     public bool CheckLineOfSight(){
00042         Vector2 playerPos = GetParent().GetNode<CharacterBody2D>("Player").Position;
00043         double distance = Mathf.Sqrt((playerPos.X - Position.X)*(playerPos.X - Position.X) +
00044             (playerPos.Y - Position.Y) *(playerPos.Y - Position.Y));
00045         if(distance < 300){
00046             return true;
00047         }
00048         return false;
00049     }
00050 }

```

9.7 TurretHealth.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class TurretHealth : Node2D
00005 {
00006     public Health health = new Health(100,100,100,100,100,100);
00007 }

```

9.8 GlobalUsings.cs

```

00001 global using Xunit;

```

9.9 Health.cs

```

00001 using Xunit.Sdk;
00002
00003 public class Health
00004 {
00005     public int _hp { get; private set; }
00006     public int _hpMax { get; private set; }
00007     public int _shield { get; private set; }
00008     public int _shieldMax { get; private set; }
00009     public int _resistancePhysical { get; private set; }
00010     public int _resistanceElectric { get; private set; }
00011     public int _vulnPhysical { get; private set; }
00012     public int _vulnElectric { get; private set; }
00013     public bool _alive { get; private set; } = true;
00014     public const int _damageBias = 12;
00015
00016     public Health(int hpMax, int shieldMax, int initialResistancePhysical, int
initialResistanceElectric, int vulnPhysical, int vulnElectric)
00017     {
00018         _hpMax = hpMax;
00019         _shieldMax = shieldMax;
00020         _hp = _hpMax;
00021         _shield = _hpMax;
00022         _resistancePhysical = initialResistancePhysical;
00023         _resistanceElectric = initialResistanceElectric;
00024         _vulnElectric = vulnElectric;
00025         _vulnPhysical = vulnPhysical;
00026     }
00027
00028     public void ApplyTrueDamage(int damage)
00029     {
00030         int damageAfterShield = Math.Max(damage - _shield, 0);
00031         _shield = Math.Max(_shield - damage, 0);
00032         _hp = _hp - damageAfterShield;
00033
00034         if (_hp <= 0)
00035         {
00036             _hp = 0;
00037             _alive = false;
00038         }
00039     }
00040     public void ApplyDamage(int damagePhysical, int damageElectric)
00041     {
00042         // line-by-line for easy debugging
00043         // doing stuff just twice downsn't warrant its own function
00044
00045         int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00046         int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00047
00048         int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00049         int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00050
00051         int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00052
00053         ApplyTrueDamage(damageTotal);
00054     }
00055
00056     public void HealHP(int amount)
00057     {
00058         _hp = Math.Min(_hp + amount, _hpMax);
00059     }
00060
00061     public void HealShield(int amount)
00062     {

```

```

00063         _shield = Math.Min(_shield + amount, _shieldMax);
00064     }
00065
00066     public void HealBoth(int healAmount)
00067     {
00068         int healthMissing = _hpMax - _hp;
00069
00070         _hp = Math.Min(_hp + healAmount, _hpMax);
00071
00072         if (healAmount > healthMissing)
00073         {
00074             _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00075         }
00076     }
00077
00078     public void DestroyShield()
00079     {
00080         _shield = 0;
00081     }
00082
00083     public void RestoreHealth()
00084     {
00085         _hp = _hpMax;
00086     }
00087
00088     public void RestoreShield()
00089     {
00090         _shield = _shieldMax;
00091     }
00092
00093     public void damageShield(int damage)
00094     {
00095         _shield = Math.Max(_shield - damage, 0);
00096     }
00097
00098     public void damageHP(int damage)
00099     {
00100         _hp = Math.Max(_hp - damage, 0);
00101         _alive = Convert.ToBoolean(_hp);
00102     }
00103
00104     public void Revive()
00105     {
00106         _hp = 1;
00107         _shield = 0;
00108         _alive = true;
00109     }

```

9.10 Health.cs

```

00001 using System;
00002
00003 public class Health
00004 {
00005     public int _hp { get; private set; }
00006     public int _hpMax { get; private set; }
00007     public int _shield { get; private set; }
00008     public int _shieldMax { get; private set; }
00009     public int _resistancePhysical { get; private set; }
00010     public int _resistanceElectric { get; private set; }
00011     public int _vulnPhysical { get; private set; }
00012     public int _vulnElectric { get; private set; }
00013     public bool _alive { get; private set; } = true;
00014     public const int _damageBias = 12;
00015
00016     public Health(int hpMax, int shieldMax, int initialResistancePhysical, int
initialResistanceElectric, int vulnPhysical, int vulnElectric)
00017     {
00018         _hpMax = hpMax;
00019         _shieldMax = shieldMax;
00020         _hp = _hpMax;
00021         _shield = _hpMax;
00022         _resistancePhysical = initialResistancePhysical;
00023         _resistanceElectric = initialResistanceElectric;
00024         _vulnElectric = vulnElectric;
00025         _vulnPhysical = vulnPhysical;
00026     }
00027
00028     public void ApplyTrueDamage(int damage)
00029     {
00030         int damageAfterShield = Math.Max(damage - _shield, 0);
00031         _shield = Math.Max(_shield - damage, 0);
00032         _hp = _hp - damageAfterShield;

```

```

00033
00034     if (_hp <= 0)
00035     {
00036         _hp = 0;
00037         _alive = false;
00038     }
00039 }
00040 public void ApplyDamage(int damagePhysical, int damageElectric)
00041 {
00042     // line-by-line for easy debugging
00043     // doing stuff just twice doesn't warrant its own function
00044
00045     int damagePhysicalAfterResistance = ((1000 - _resistancePhysical) * damagePhysical) / 1000;
00046     int damageElectricAfterResistance = ((1000 - _resistanceElectric) * damageElectric) / 1000;
00047
00048     int damagePhysicalAfterVuln = ((1000 + _vulnPhysical) * damagePhysicalAfterResistance) / 1000;
00049     int damageElectricAfterVuln = ((1000 + _vulnElectric) * damageElectricAfterResistance) / 1000;
00050
00051     int damageTotal = damagePhysicalAfterVuln + damageElectricAfterVuln;
00052
00053     ApplyTrueDamage(damageTotal);
00054 }
00055
00056 public void HealHP(int amount)
00057 {
00058     _hp = Math.Min(_hp + amount, _hpMax);
00059 }
00060
00061 public void HealShield(int amount)
00062 {
00063     _shield = Math.Min(_shield + amount, _shieldMax);
00064 }
00065
00066 public void HealBoth(int healAmount)
00067 {
00068     int healthMissing = _hpMax - _hp;
00069
00070     _hp = Math.Min(_hp + healAmount, _hpMax);
00071
00072     if (healAmount > healthMissing)
00073     {
00074         _shield = Math.Min(_shield + healAmount - healthMissing, _shieldMax);
00075     }
00076 }
00077
00078 public void DestroyShield()
00079 {
00080     _shield = 0;
00081 }
00082
00083 public void RestoreHealth()
00084 {
00085     _hp = _hpMax;
00086 }
00087
00088 public void RestoreShield()
00089 {
00090     _shield = _shieldMax;
00091 }
00092
00093 public void damageShield(int damage)
00094 {
00095     _shield = Math.Max(_shield - damage, 0);
00096 }
00097
00098 public void damageHP(int damage)
00099 {
00100     _hp = Math.Max(_hp - damage, 0);
00101     _alive = Convert.ToBoolean(_hp);
00102 }
00103
00104 public void Revive()
00105 {
00106     _hp = 1;
00107     _shield = 0;
00108     _alive = true;
00109 }

```

9.11 UnitTest1.cs

```

00001 namespace TestProject1;
00002 using Xunit;
00003 using Xunit.Abstractions;

```

```

00004
00005 public class UnitTest1
00006 {
00007     private readonly ITestOutputHelper _testLogger;
00008     public UnitTest1(ITestOutputHelper testLogger)
00009     {
00010         this._testLogger = testLogger;
00011     }
00012
00013     [Fact]
00014     public void Health_Initialization_Test()
00015     {
00016         int initialResistancePhysical = 100;
00017         int initialResistanceElectric = 300;
00018         int initialVulnPhysical = 0;
00019         int initialVulnElectric = 200;
00020
00021         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00022             initialVulnPhysical, initialVulnElectric);
00023
00024         Assert.Equal(1000, health._hpMax);
00025         Assert.Equal(1000, health._hp);
00026         Assert.Equal(1000, health._shield);
00027         Assert.Equal(initialResistancePhysical, health._resistancePhysical);
00028         Assert.Equal(initialResistanceElectric, health._resistanceElectric);
00029         Assert.Equal(initialVulnPhysical, health._vulnPhysical);
00030         Assert.Equal(initialVulnElectric, health._vulnElectric);
00031         Assert.True(health._alive);
00032     }
00033
00034     [Theory]
00035     [InlineData(1000, 1000, 200, 1000, 800, true)] //Damage only to Shield
00036     [InlineData(1000, 1000, 1000, 1000, 0, true)] //Exactly destroy shield
00037     [InlineData(1000, 1000, 1200, 800, 0, true)] //Destroy shield with extra damage to health
00038     [InlineData(1000, 1000, 3000, 0, 0, false)] //Overkill
00039     [InlineData(1000, 1000, 1000, 2000, 0, 0, false)] //Exact deadly damage
00040     [InlineData(1000, 1000, 0, 1000, 1000, true)] //No damage
00041     public void True_Damage_Test(
00042         int startHP, int startShield, int appliedDamage,
00043         int expectedHp, int expectedShield, bool expectedAlive)
00044     {
00045         // Arrange
00046         Health health = new Health(startHP, startShield, 1000, 1000, 0, 0);
00047
00048         // Act
00049         health.ApplyTrueDamage(appliedDamage);
00050
00051         // Assert
00052         Assert.Equal(expectedHp, health._hp);
00053         Assert.Equal(expectedShield, health._shield);
00054         Assert.Equal(expectedAlive, health._alive);
00055     }
00056
00057     [Theory]
00058     [InlineData(100, 100, 0, 0, 9000, 9000, 0, 0, false)] //Overkill
00059     [InlineData(100, 100, 100, 100, 300, 300, 1000, 406, true)] //Calculation Test + no shield break
00060     [InlineData(500, 500, 1000, 1000, 500, 500, 1000, 0, true)] //Exactly destroy shield
00061     [InlineData(500, 500, 1000, 1000, 750, 750, 500, 0, true)] //Destroy shield with extra damage to
health
00062     [InlineData(500, 500, 1000, 1000, 1000, 1000, 0, 0, false)] //Exactly deadly damage
00063     [InlineData(1000, 1000, 1000, 1000, 9000, 9000, 1000, 1000, true)] //Resistance blocks damage
00064     public void ApplyDamage_Test(
00065         int initialResistancePhysical, int initialResistanceElectric,
00066         int initialVulnPhysical, int initialVulnElectric,
00067         int damagePhysical, int damageElectric,
00068         int expectedHp, int expectedShield, bool expectedAlive)
00069     {
00070         // Arrange
00071         Health health = new Health(1000, 1000, initialResistancePhysical, initialResistanceElectric,
00072             initialVulnPhysical, initialVulnElectric);
00073
00074         // Act
00075         health.ApplyDamage(damagePhysical, damageElectric);
00076
00077         // Assert
00078         Assert.Equal(expectedHp, health._hp);
00079         Assert.Equal(expectedShield, health._shield);
00080         Assert.Equal(expectedAlive, health._alive);
00081     }
00082
00083     [Theory]
00084     [InlineData(1000, 1000, 0, 0, 0, 1000, 1000, true)] // heal nothing
00085     [InlineData(1000, 1000, 1000, 500, 0, 250, 750, 1000, true)] // heal HP only
00086     [InlineData(1000, 1000, 500, 0, 1000, 1000, 1000, true)] // overheal HP only
00087     [InlineData(1000, 1000, 0, 500, 250, 1000, 750, true)] // heal Shield only
00088     [InlineData(1000, 1000, 0, 500, 1000, 1000, 1000, true)] // overheal Shield only
00089     [InlineData(1000, 1000, 500, 500, 750, 1000, 750, true)] // partial heal both

```

```

00090     [InlineData(1000, 1000, 500, 500, 1000, 1000, 1000, true)] // heal both
00091     [InlineData(1000, 1000, 500, 500, 2000, 1000, 1000, true)] // overheal both
00092     public void HealBoth_Test(
00093         int initialHP, int initialShield,
00094         int damageHP, int damageShield, int healAmount,
00095         int expectedHp, int expectedShield, bool expectedAlive)
00096     {
00097         // Arrange
00098         Health health = new Health(initialHP, initialShield, 0, 0, 0, 0);
00099
00100         // Act
00101         health.damageHP(damageHP);
00102         health.damageShield(damageShield);
00103
00104         health.HealBoth(healAmount);
00105
00106         // Assert
00107         Assert.Equal(expectedHp, health._hp);
00108         Assert.Equal(expectedShield, health._shield);
00109         Assert.Equal(expectedAlive, health._alive);
00110     }
00111 }

```

9.12 InteractionArea.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class InteractionArea : Area2D
00005 {
00006     [Export]
00007     private String interactionLabel = "none";
00008     [Export]
00009     private String interactionValue = "none";
00010     [Export]
00011     private String interactionType = "none";
00012
00013     public String GetLabel(){
00014         return interactionLabel;
00015     }
00016     public String GetInteractType(){
00017         return interactionType;
00018     }
00019     public String GetValue(){
00020         return interactionValue;
00021     }
00022     public void SetLabel(String newLabel){
00023         interactionLabel = newLabel;
00024     }
00025     public void SetInteractType(String newType){
00026         interactionType = newType;
00027     }
00028     public void SetValue(String newValue){
00029         interactionValue = newValue;
00030     }
00031
00032     public override void _Ready(){
00033         GetNode<NinePatchRect>("DialogueManager").Visible = false;
00034     }
00035
00036
00037     public void StartDialogue(String dialogueNumber){
00038         GetNode<NinePatchRect>("DialogueManager").Visible = true;
00039         switch(dialogueNumber){
00040             case "0":
00041                 DisplayDialogue("Info!", "Welcome to Heidelberg Dynamics! You are L4lk4, and need to
00042 escape this facility." + " Use [W][A][S][D] to move to the next room. Press [E] to
00043 continue.");
00044                 break;
00045             case "1":
00046                 DisplayDialogue("Info!", "You need to be careful. This facility is full of enemy
00047 turrets that will " + " shoot you on sight. Use [LMB] to shoot your weapon.");
00048                 break;
00049             case "2":
00050                 DisplayDialogue("Info!", "You need to collect a keycard to open the door to the next
00051 level.");
00052                 break;
00053             case "3":
00054                 DisplayDialogue("Info!", "You escaped! You can now enjoy your freedom.");
00055                 break;
00056             default:

```

```

00055         EndDialogue();
00056         break;
00057     }
00058 }
00059
00060 public void DisplayDialogue(String name,String text){
00061     GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Name").Text = name;
00062     GetNode<NinePatchRect>("DialogueManager").GetNode<RichTextLabel>("Text").Text = text;
00063 }
00064
00065 public void EndDialogue(){
00066     GetNode<NinePatchRect>("DialogueManager").Visible = false;
00067     QueueFree();
00068 }
00069
00070 }
00071

```

9.13 level_4.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class level_4 : Node2D
00005 {
00006     public override void _Ready(){
00007         GetNode<Player>("Player").SetExtraCard(false);
00008         GetNode<StaticBody2D>("keycard2").GetNode<InteractionArea>("InteractionArea").SetValue("keycard2");
00009     }
00010 }

```

9.14 explosion.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class explosion : AnimatedSprite2D
00005 {
00006     public override void _Ready(){
00007         Position = new Vector2(0,0);
00008         //Position = GetParent().GlobalPosition;
00009     }
00010
00011     private void _on_animation_finished(){
00012         QueueFree();
00013     }
00014 }

```

9.15 projectile.cs

```

00001 using Godot;
00002 using System;
00003
00004 public partial class projectile : RigidBody2D
00005 {
00006     private PackedScene explosion = GD.Load<PackedScene>("res://projectiles/explosion.tscn");
00007     private Timer timer;
00008
00009     public override void _Ready(){
00010         timer = new Timer();
00011         this.AddChild(timer);
00012         timer.WaitTime = 0.34;
00013         timer.OneShot = true;
00014         timer.Start();
00015     }
00016
00017     private void _on_body_entered(Node body){
00018         if(!body.IsInGroup("entity")){
00019             QueueFree();
00020         }
00021         else if(body.IsInGroup("player")){
00022             QueueFree();
00023             body.GetNode<PlayerHealth>("Health").health.ApplyDamage(50,50);
00024             body.GetNode<AudioStreamPlayer>("HitSound").Play();

```

```
00025     }
00026     else if (body.IsInGroup("boss")) {
00027         QueueFree();
00028         body.GetNode<BossHealth>("Health").health.ApplyDamage(50, 50);
00029         body.GetNode<AudioStreamPlayer>("HitSound").Play();
00030     }
00031     else {
00032         QueueFree();
00033         body.GetNode<TurretHealth>("Health").health.ApplyDamage(50, 50);
00034         body.GetNode<AudioStreamPlayer>("HitSound").Play();
00035     }
00036 }
00037
00038 public override void _PhysicsProcess(double delta) {
00039     if (timer.IsStopped())
00040         QueueFree();
00041 }
00042
00043
00044 }
```

Index

- [_PhysicsProcess](#)
 - [Bossturret, 18](#)
 - [Player, 41](#)
 - [projectile, 47](#)
 - [Turret, 49](#)
 - [_Ready](#)
 - [DialogueManager, 21](#)
 - [explosion, 23](#)
 - [InteractionArea, 35](#)
 - [level_4, 38](#)
 - [Player, 41](#)
 - [projectile, 47](#)
 - [Turret, 49](#)
 - [_alive](#)
 - [Health, 31](#)
 - [_damageBias](#)
 - [Health, 31](#)
 - [_hp](#)
 - [Health, 31](#)
 - [_hpMax](#)
 - [Health, 31](#)
 - [_resistanceElectric](#)
 - [Health, 31](#)
 - [_resistancePhysical](#)
 - [Health, 31](#)
 - [_shield](#)
 - [Health, 32](#)
 - [_shieldMax](#)
 - [Health, 32](#)
 - [_vulnElectric](#)
 - [Health, 32](#)
 - [_vulnPhysical](#)
 - [Health, 32](#)
- [ApplyDamage](#)
 - [Health, 26](#)
- [ApplyDamage_Test](#)
 - [TestProject1.UnitTest1, 53](#)
- [ApplyTrueDamage](#)
 - [Health, 27](#)
- [BossHealth, 15](#)
 - [health, 17](#)
- [Bossturret, 17](#)
 - [_PhysicsProcess, 18](#)
 - [CheckLineOfSight, 19](#)
 - [Fire1, 19](#)
 - [Fire2, 19](#)
- [characters/BossHealth.cs, 57](#)
- [characters/Bossturret.cs, 57](#)
- [characters/NinePatchRect.cs, 58](#)
- [characters/Player.cs, 58](#)
- [characters/PlayerHealth.cs, 61](#)
- [characters/Turret.cs, 61](#)
- [characters/TurretHealth.cs, 62](#)
- [CheckLineOfSight](#)
 - [Bossturret, 19](#)
 - [Turret, 49](#)
- [damageHP](#)
 - [Health, 27](#)
- [damageShield](#)
 - [Health, 28](#)
- [DestroyShield](#)
 - [Health, 28](#)
- [DialogueManager, 20](#)
 - [_Ready, 21](#)
 - [DisplayDialogue, 21](#)
 - [EndDialogue, 21](#)
 - [StartDialogue, 21](#)
- [DisplayDialogue](#)
 - [DialogueManager, 21](#)
 - [InteractionArea, 35](#)
- [EndDialogue](#)
 - [DialogueManager, 21](#)
 - [InteractionArea, 35](#)
- [EndGame](#)
 - [Player, 41](#)
- [ExecuteInteract](#)
 - [Player, 42](#)
- [explosion, 22](#)
 - [_Ready, 23](#)
- [Fire](#)
 - [Player, 42](#)
 - [Turret, 49](#)
- [Fire1](#)
 - [Bossturret, 19](#)
- [Fire2](#)
 - [Bossturret, 19](#)
- [Game Systems, tested using Xunit, 1, 3](#)
- [game-systems/TestProject1/GlobalUsings.cs, 62](#)
- [game-systems/TestProject1/Health.cs, 62](#)
- [game-systems/TestProject1/UnitTest1.cs, 64](#)
- [GetInteractType](#)
 - [InteractionArea, 35](#)
- [GetLabel](#)

- InteractionArea, 35
- GetPosition
 - Player, 43
- GetValue
 - InteractionArea, 35
- HealBoth
 - Health, 28, 29
- HealBoth_Test
 - TestProject1.UnitTest1, 53
- HealHP
 - Health, 29
- HealShield
 - Health, 29
- Health, 24
 - _alive, 31
 - _damageBias, 31
 - _hp, 31
 - _hpMax, 31
 - _resistanceElectric, 31
 - _resistancePhysical, 31
 - _shield, 32
 - _shieldMax, 32
 - _vulnElectric, 32
 - _vulnPhysical, 32
 - ApplyDamage, 26
 - ApplyTrueDamage, 27
 - damageHP, 27
 - damageShield, 28
 - DestroyShield, 28
 - HealBoth, 28, 29
 - HealHP, 29
 - HealShield, 29
 - Health, 26
 - RestoreHealth, 30
 - RestoreShield, 30
 - Revive, 30
- health
 - BossHealth, 17
 - PlayerHealth, 46
 - TurretHealth, 52
- Health_Initialization_Test
 - TestProject1.UnitTest1, 54
- IncreaseBossCount
 - Player, 43
- InteractionArea, 33
 - _Ready, 35
 - DisplayDialogue, 35
 - EndDialogue, 35
 - GetInteractType, 35
 - GetLabel, 35
 - GetValue, 35
 - SetInteractType, 36
 - SetLabel, 36
 - SetValue, 36
 - StartDialogue, 36
- level_4, 37
 - _Ready, 38
- levels/level_4.cs, 67
- Player, 39
 - _PhysicsProcess, 41
 - _Ready, 41
 - EndGame, 41
 - ExecuteInteract, 42
 - Fire, 42
 - GetPosition, 43
 - IncreaseBossCount, 43
 - SetExtraCard, 43
 - SetMoveable, 43
 - UpdateAnimation, 43
- PlayerHealth, 44
 - health, 46
- projectile, 46
 - _PhysicsProcess, 47
 - _Ready, 47
- projectiles/explosion.cs, 67
- projectiles/projectile.cs, 67
- RestoreHealth
 - Health, 30
- RestoreShield
 - Health, 30
- Revive
 - Health, 30
- SetExtraCard
 - Player, 43
- SetInteractType
 - InteractionArea, 36
- SetLabel
 - InteractionArea, 36
- SetMoveable
 - Player, 43
- SetValue
 - InteractionArea, 36
- StartDialogue
 - DialogueManager, 21
 - InteractionArea, 36
- systems/TestProject1/Health.cs, 63
- TestProject1, 13
- TestProject1.UnitTest1, 52
 - ApplyDamage_Test, 53
 - HealBoth_Test, 53
 - Health_Initialization_Test, 54
 - True_Damage_Test, 54
 - UnitTest1, 53
- True_Damage_Test
 - TestProject1.UnitTest1, 54
- Turret, 48
 - _PhysicsProcess, 49
 - _Ready, 49
 - CheckLineOfSight, 49
 - Fire, 49
- TurretHealth, 50

health, [52](#)

UnitTest1

TestProject1.UnitTest1, [53](#)

UpdateAnimation

Player, [43](#)